

Clase 8

Desarrollando servidores

IIC2513 - Tecnologías y Aplicaciones Web

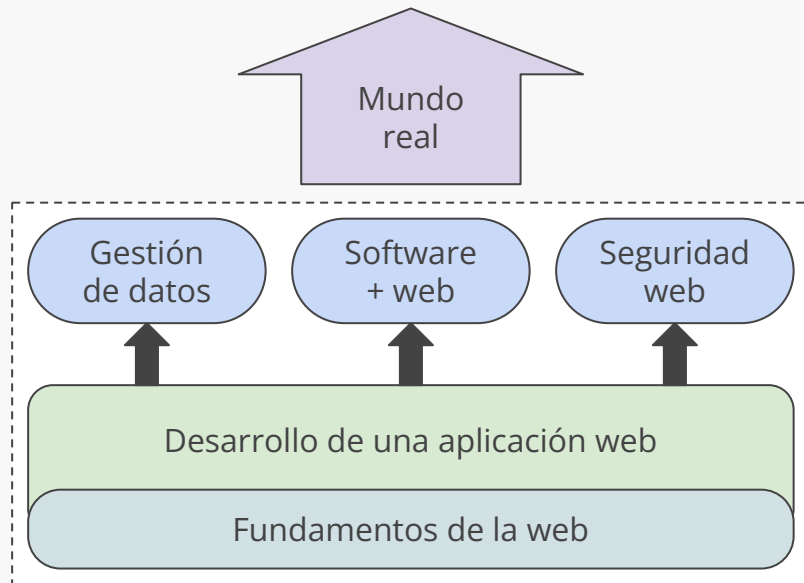
Antonio Ossa Guerra
aaossa@ing.puc.cl

1. Ánimo con la Tarea 1
2. Recuerden la ECA

Anuncios

Contenidos del curso

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real

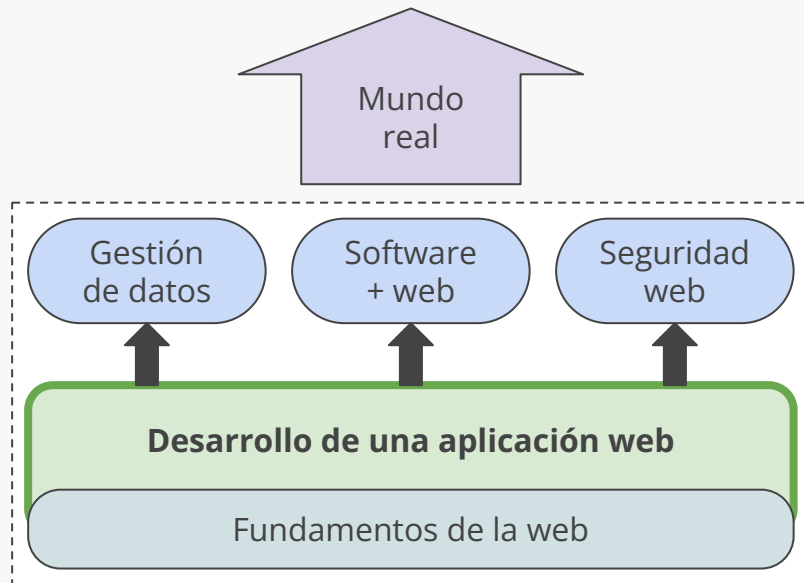


Contenidos del curso

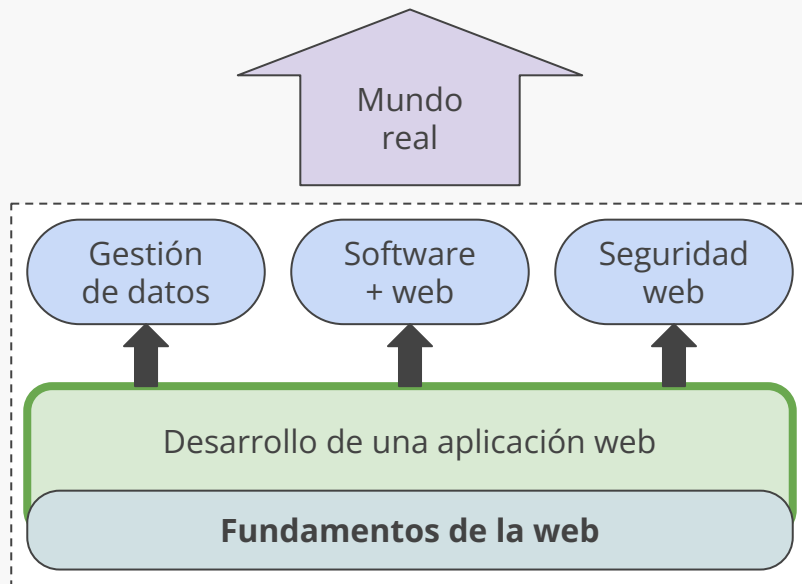
Desarrollo de una aplicación web

Veremos herramientas como React, Node.js, **Koa** y **Express**, y técnicas como CSS avanzado y el uso de *Design Systems*

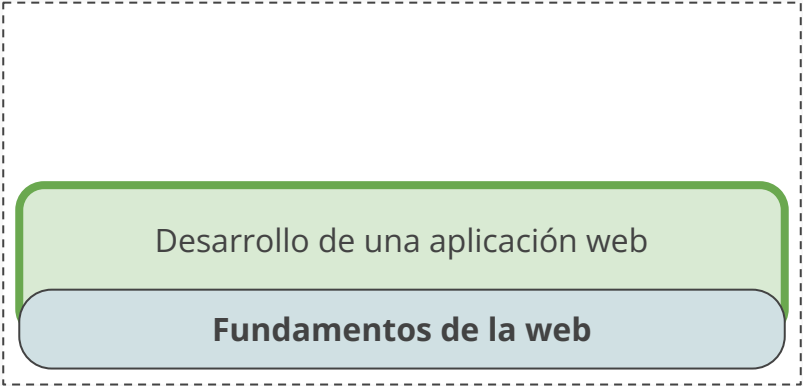
... para desarrollar aplicaciones modernas tanto en el lado del cliente (front-end) como en el lado del **servidor** (back-end)



Contenidos del curso



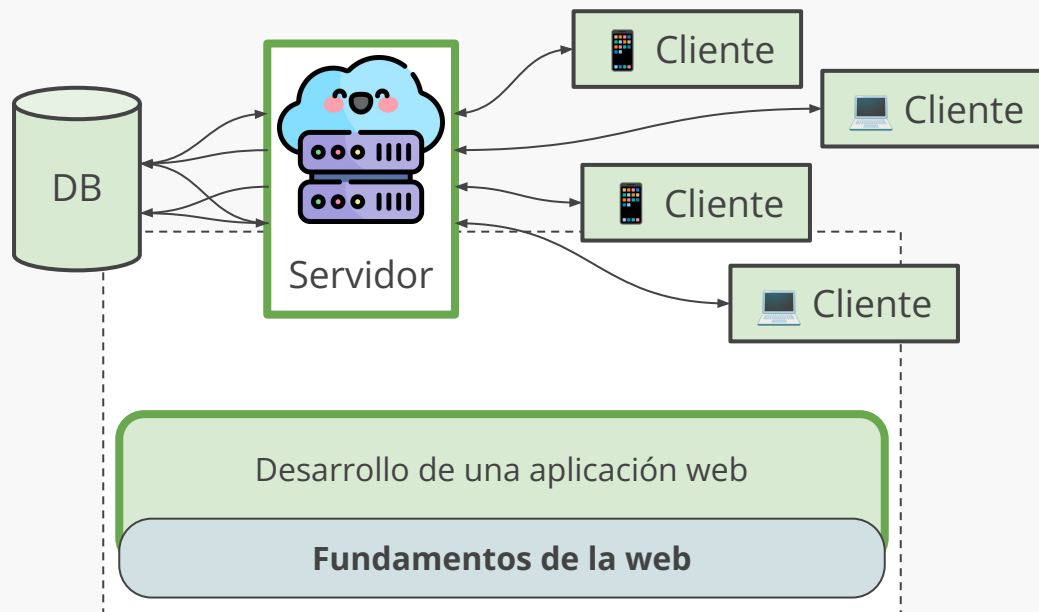
Contenidos del curso



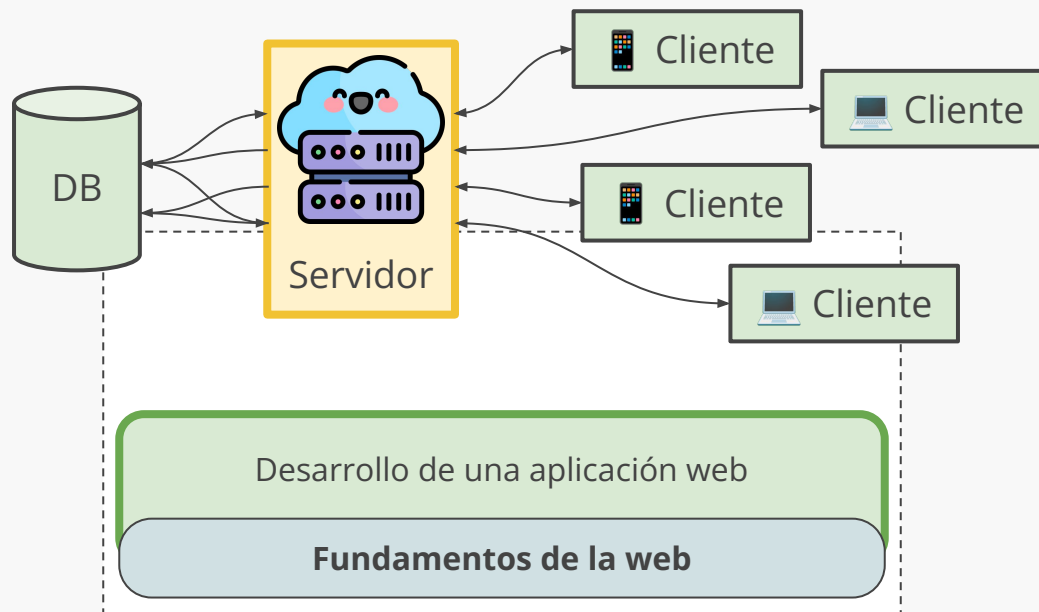
Desarrollo de una aplicación web

Fundamentos de la web

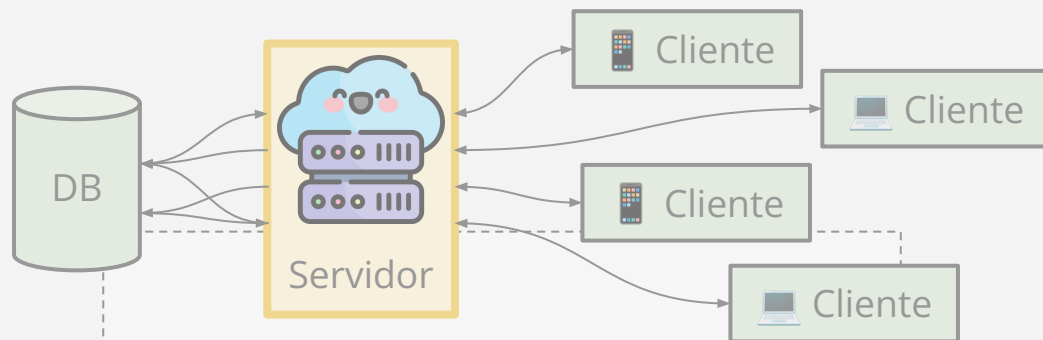
Motivación



Motivación



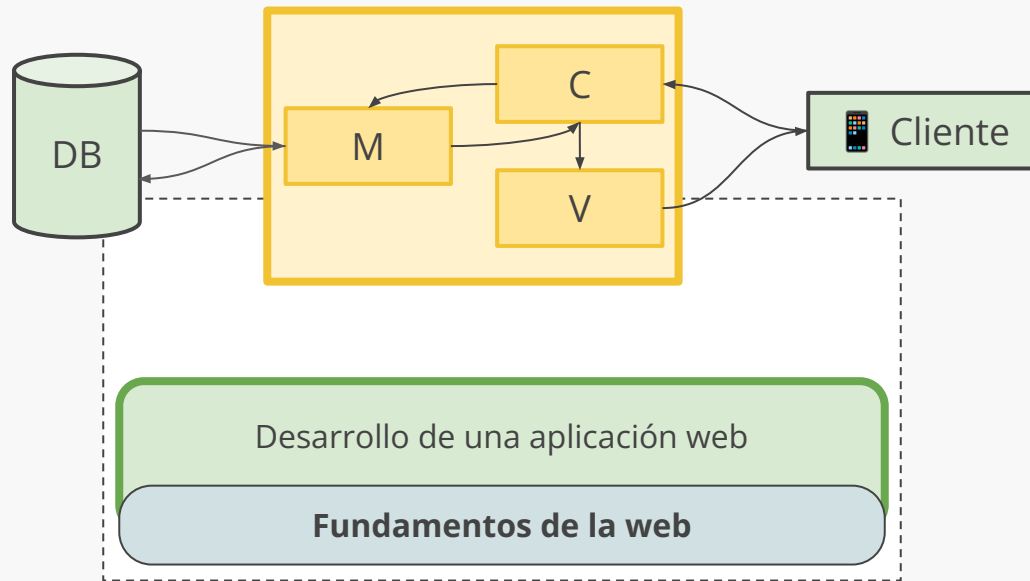
Motivación



Pregunta#¿Cómo implementamos un servidor?

Fundamentos de la web

Motivación



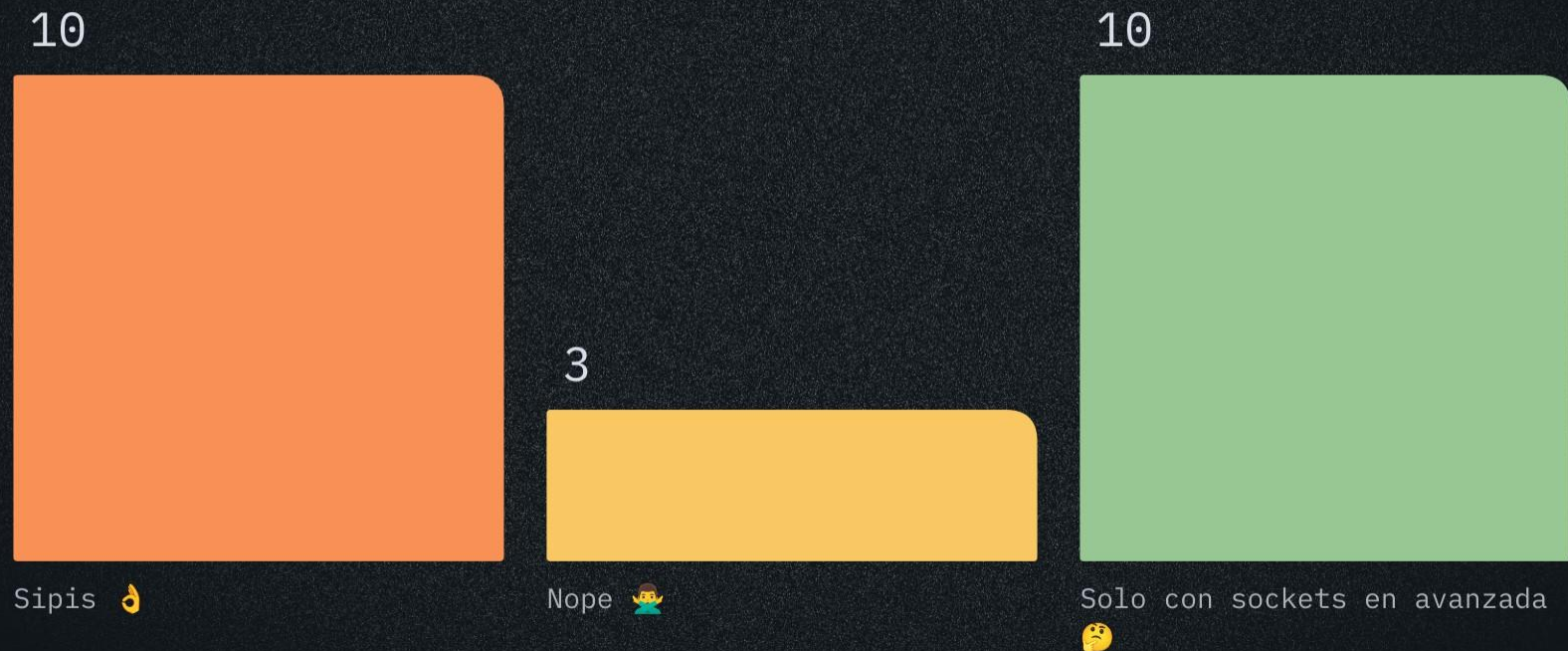
Menti*#Hablemos de servidores*

¿Cómo van con la Tarea 1?



Hablemos del contenido

¿Has implementado algún servidor?





Agenda

🌟 Cliente y servidor

Frameworks web

Implementando MVC

Evolución cliente-servidor

Cliente



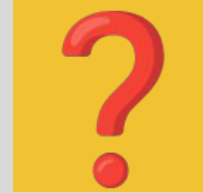
HTTP *request*



HTTP *response*



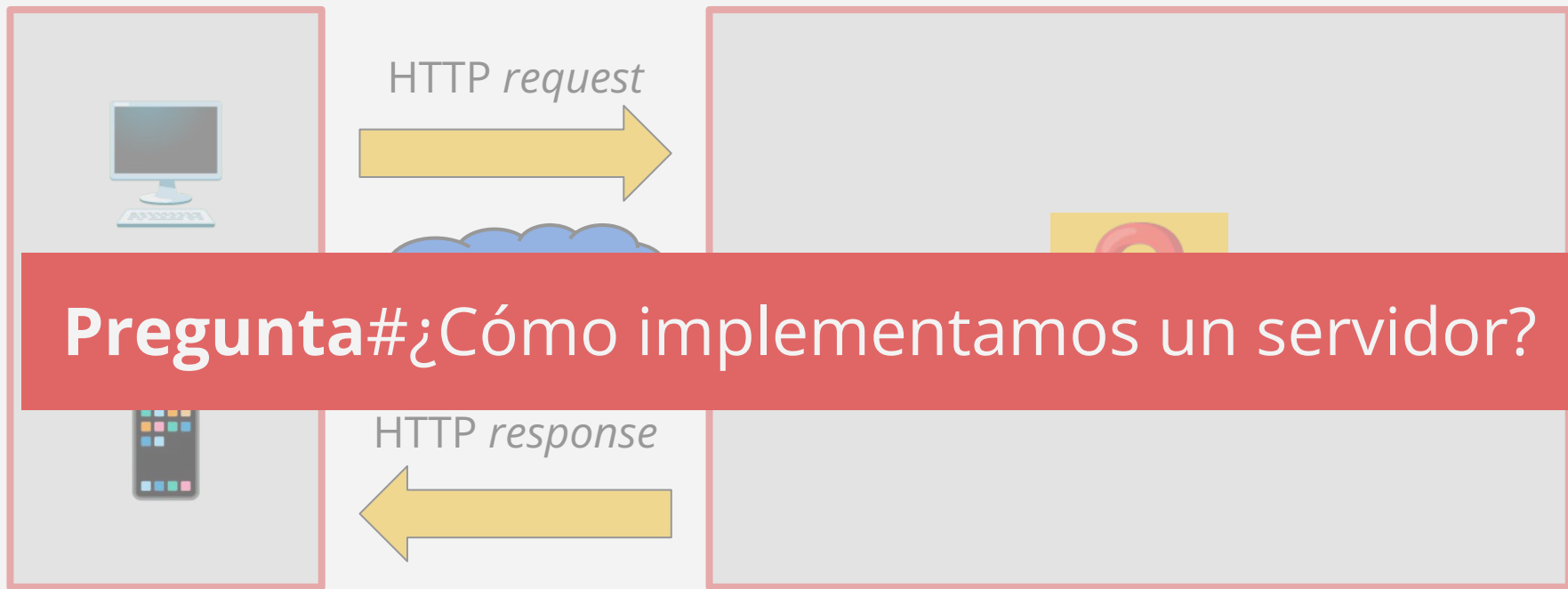
Servidor



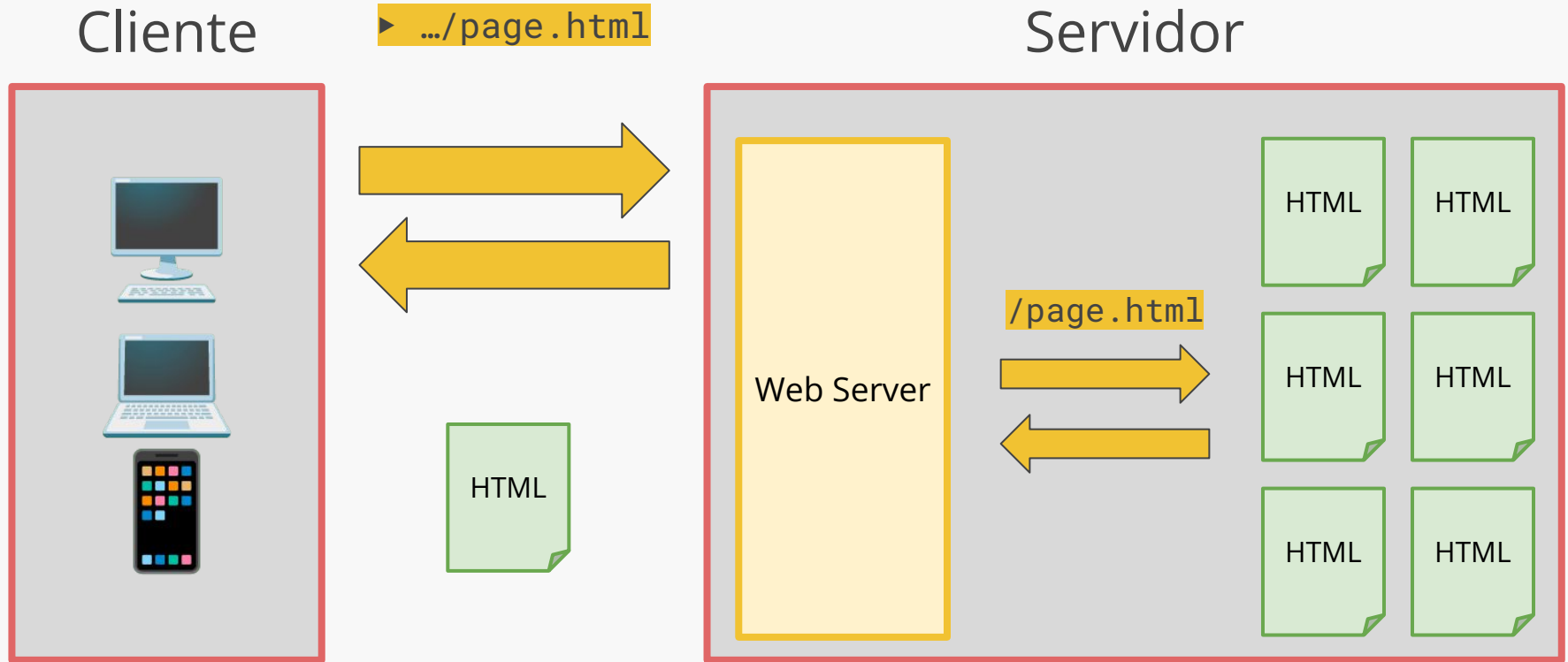
Evolución cliente-servidor

Cliente

Servidor



Inicios de la web (1990-1993)



Servidor web (*web server*)

Web Server

NGINX

 APACHE
HTTP SERVER PROJECT

Capa de software que recibe y entrega contenido **transmitiendo datos** entre el navegador del usuario (cliente) y el sistema

Provee la capa lógica que determina cómo el software maneja cada **solicitud**, **conexiones** y **tráfico**

Apache HTTP Server y NGINX



Software open source, mantenido por la [Apache Software Foundation](#), que procesa solicitudes y sirve contenido vía HTTP

Ofrece un sistema de módulos con alta performance y flexibilidad

Lanzado en 1995, fue el servidor web dominante hasta hace poco

Se inició su desarrollo en 2002 para responder al problema [C10k](#) (manejar una gran cantidad de clientes al mismo tiempo)

Su funcionamiento se basa en una arquitectura asíncrona y basada en eventos

Creado en 2002, actualmente es el servidor web más popular

Apache vs. NGINX

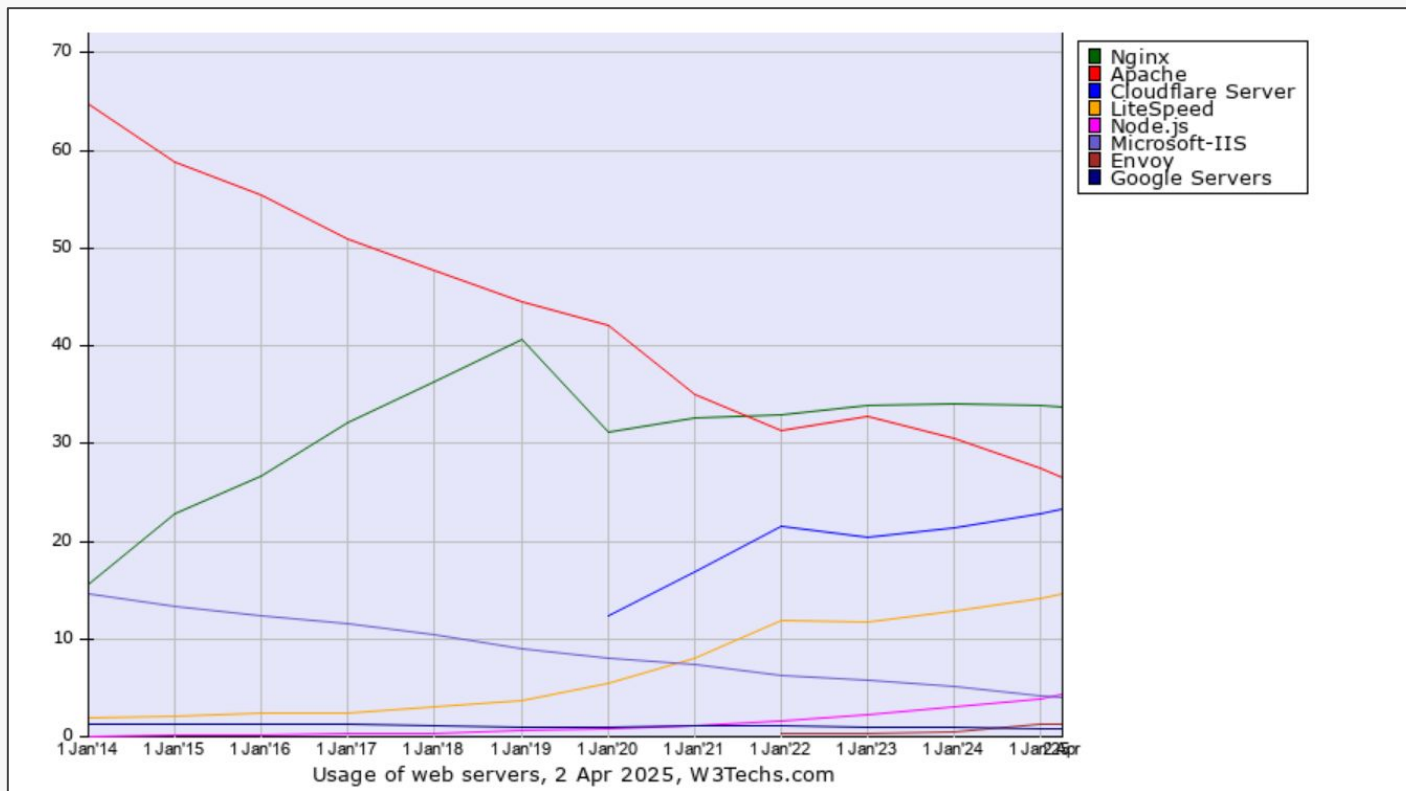


Figura: [Historical yearly trends in the usage statistics of web servers, March 2025](#)

Apache vs. NGINX

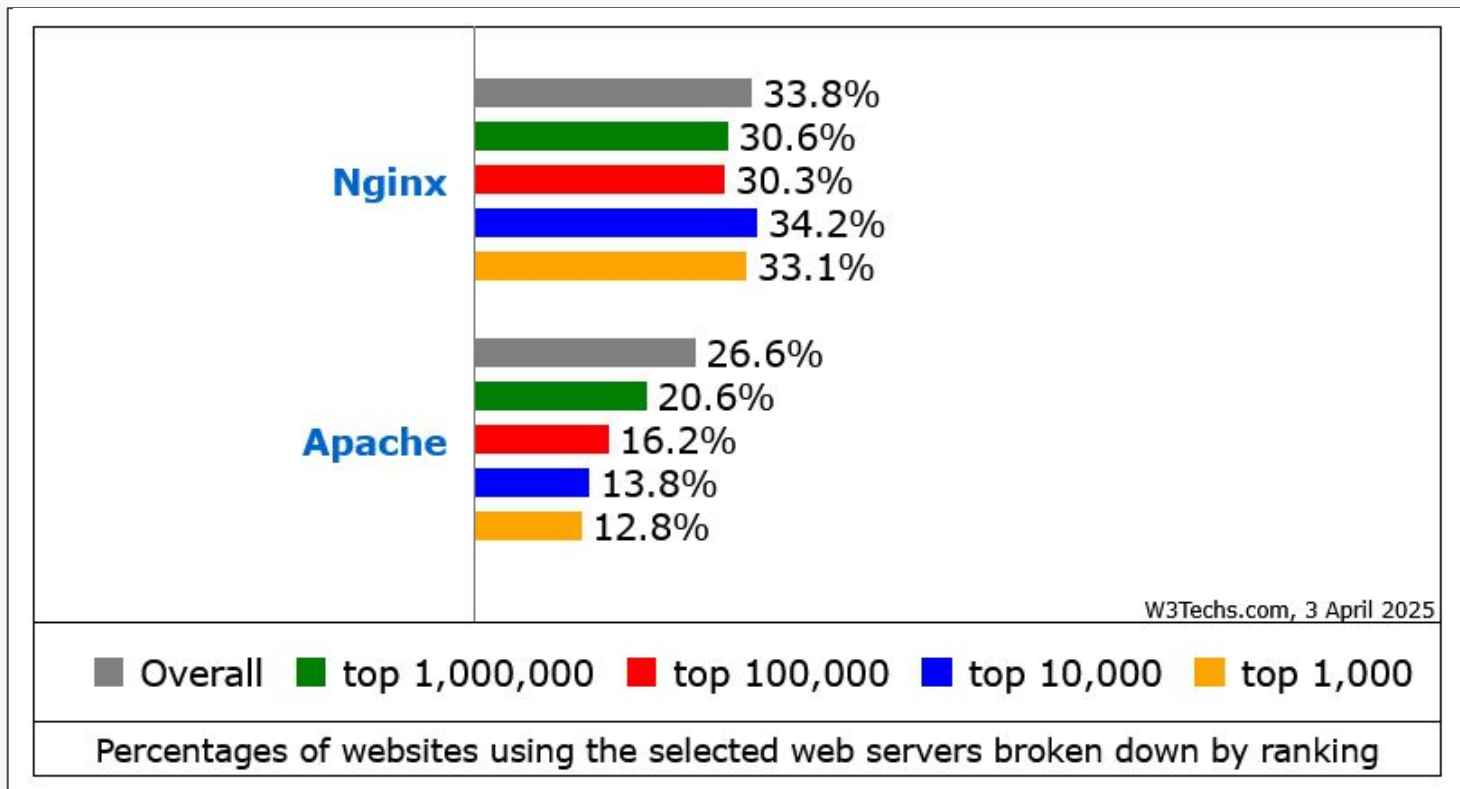


Figura: [Comparison of the usage statistics of Nginx vs. Apache for websites](#)

(Apache vs. NGINX vs. Cloudflare Server)

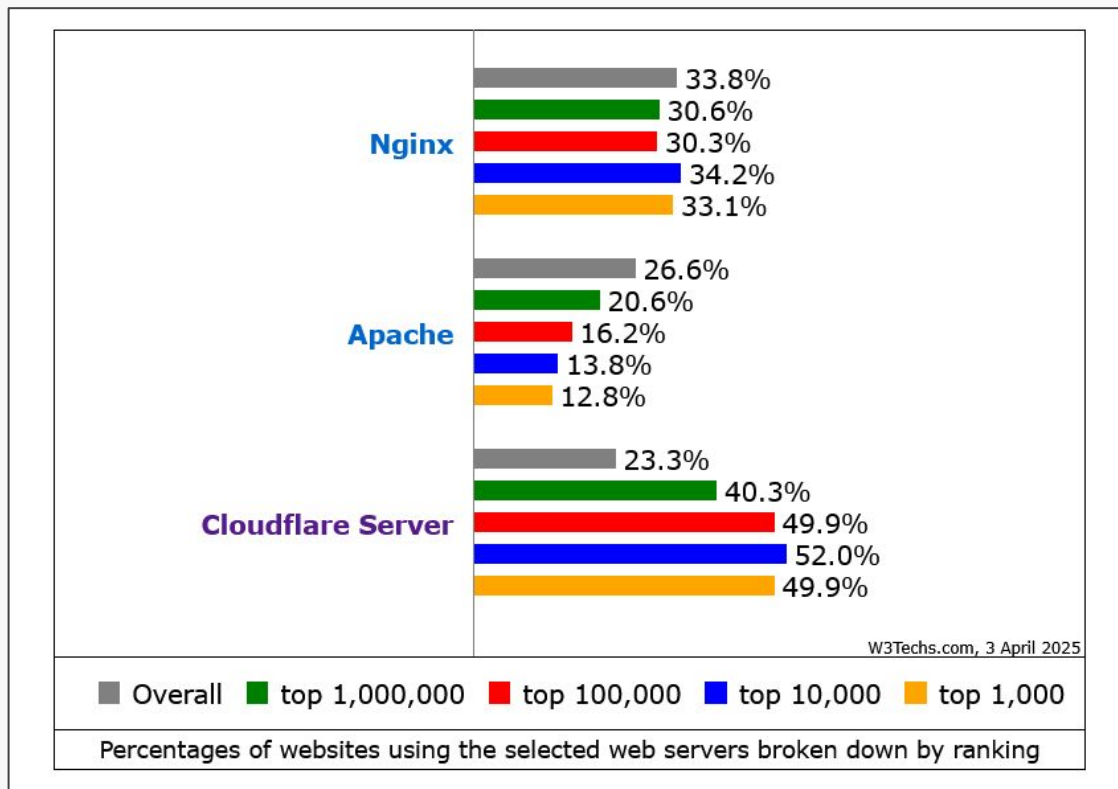


Figura: [Nginx vs. Apache vs. Cloudflare Server usage statistics](#)

(Apache vs. NGINX vs. Cloudflare Server)

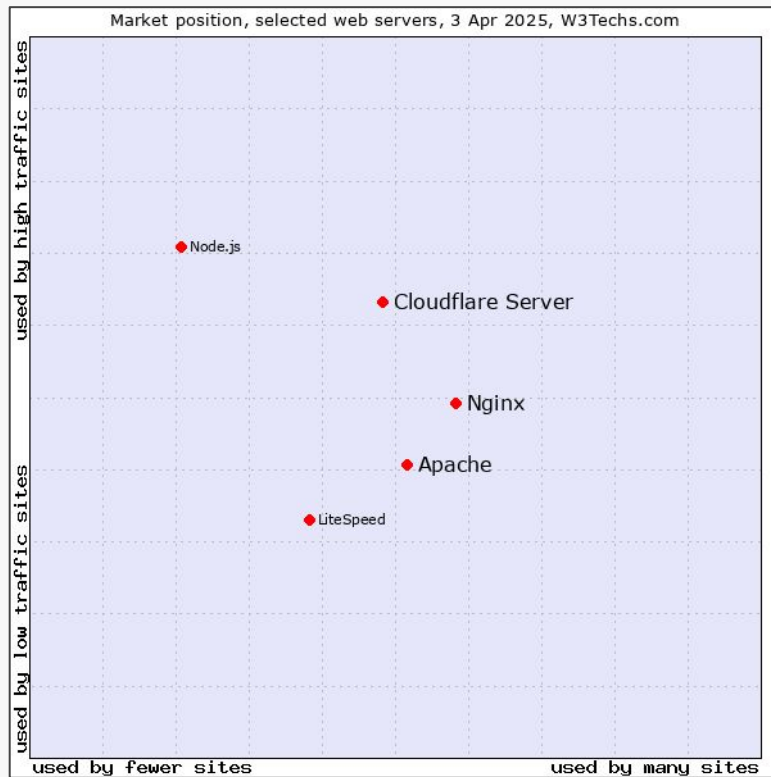
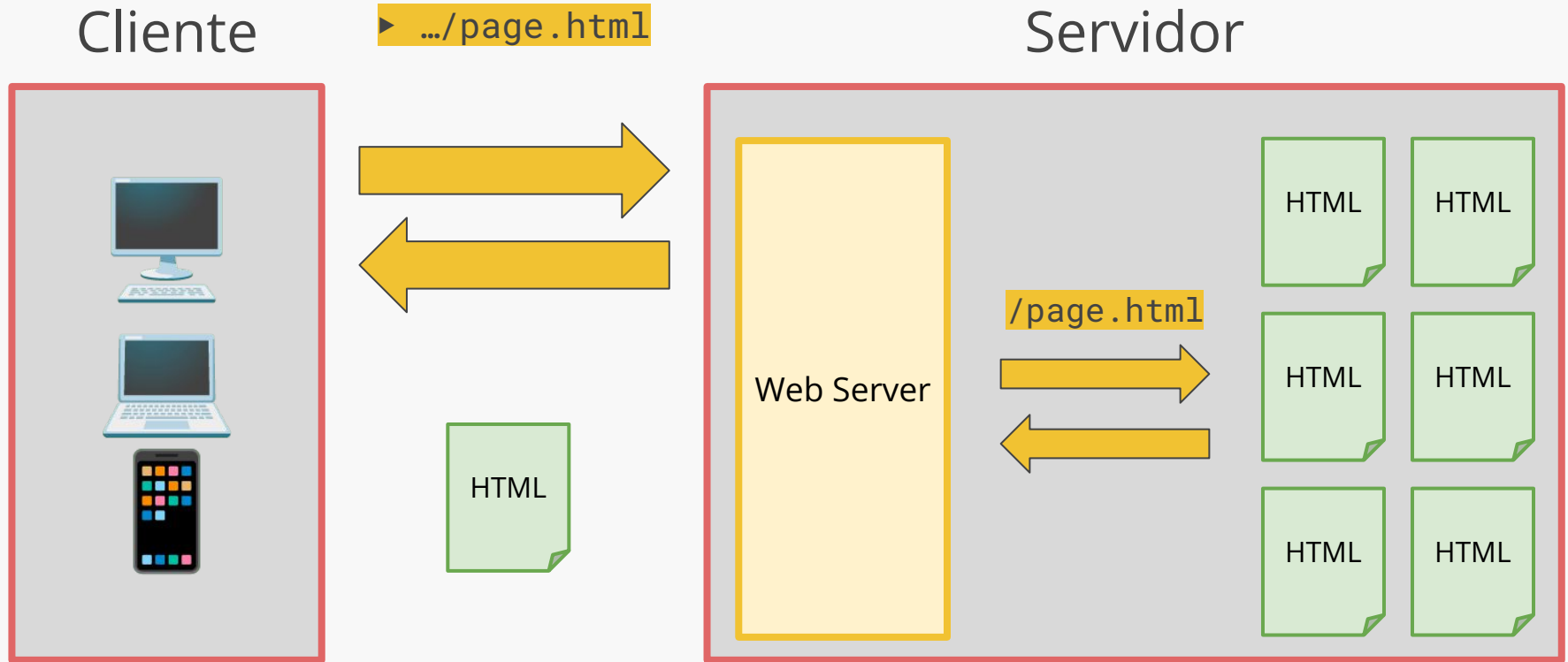
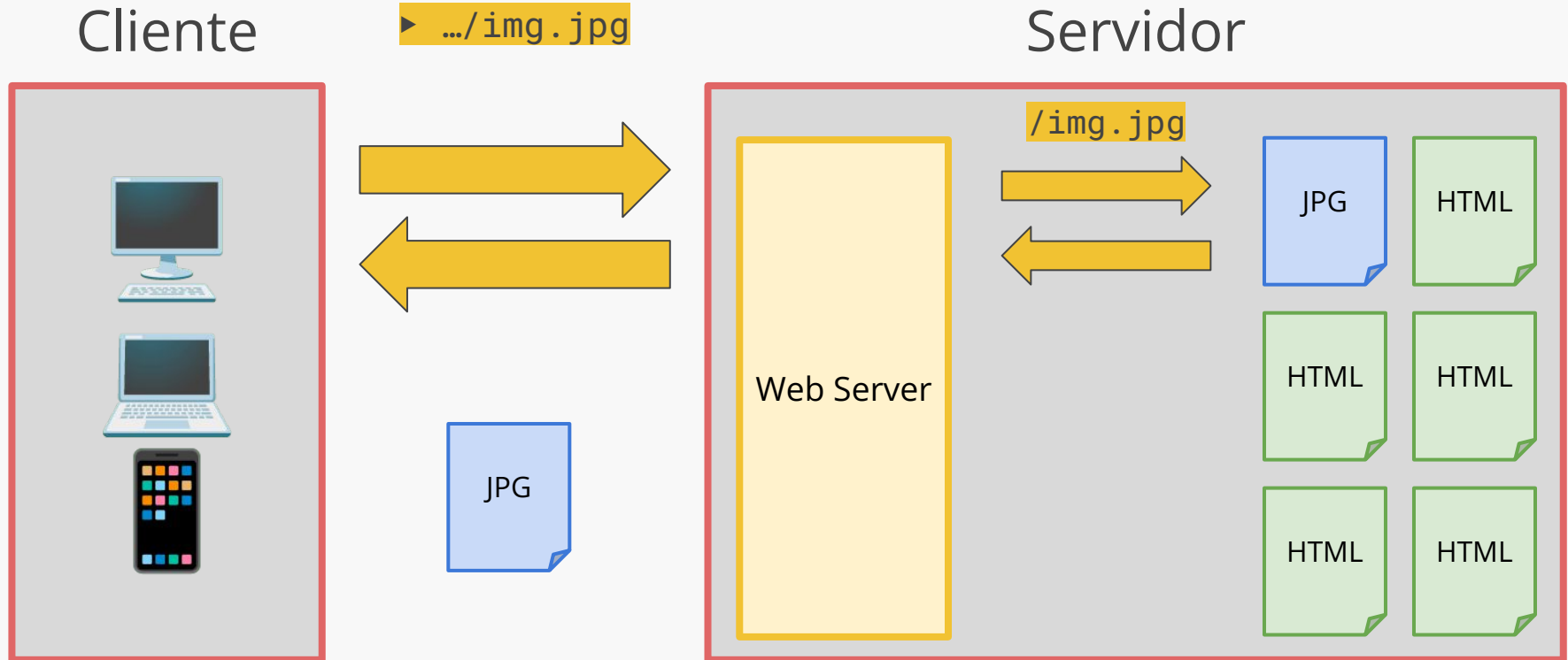


Figura: [Nginx vs. Apache vs. Cloudflare Server usage statistics](#)

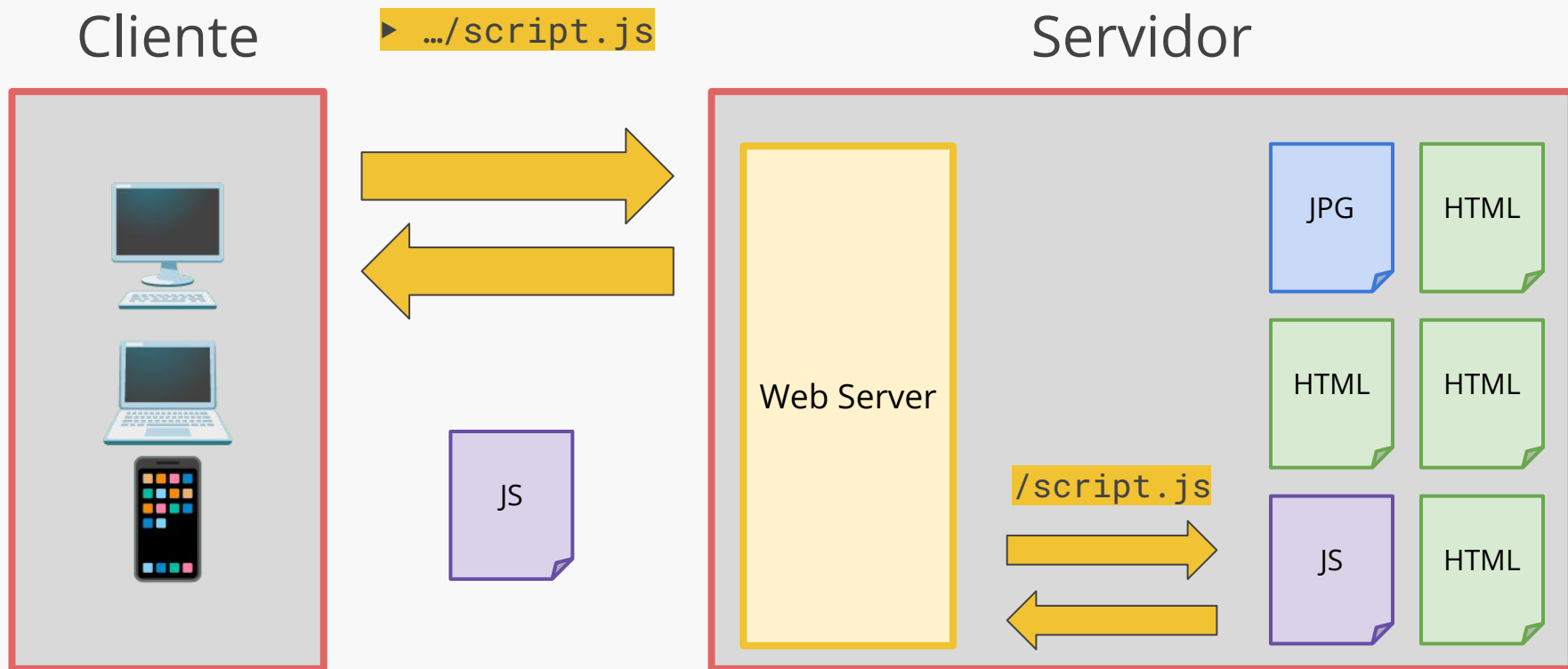
Inicios de la web (1990-1993)



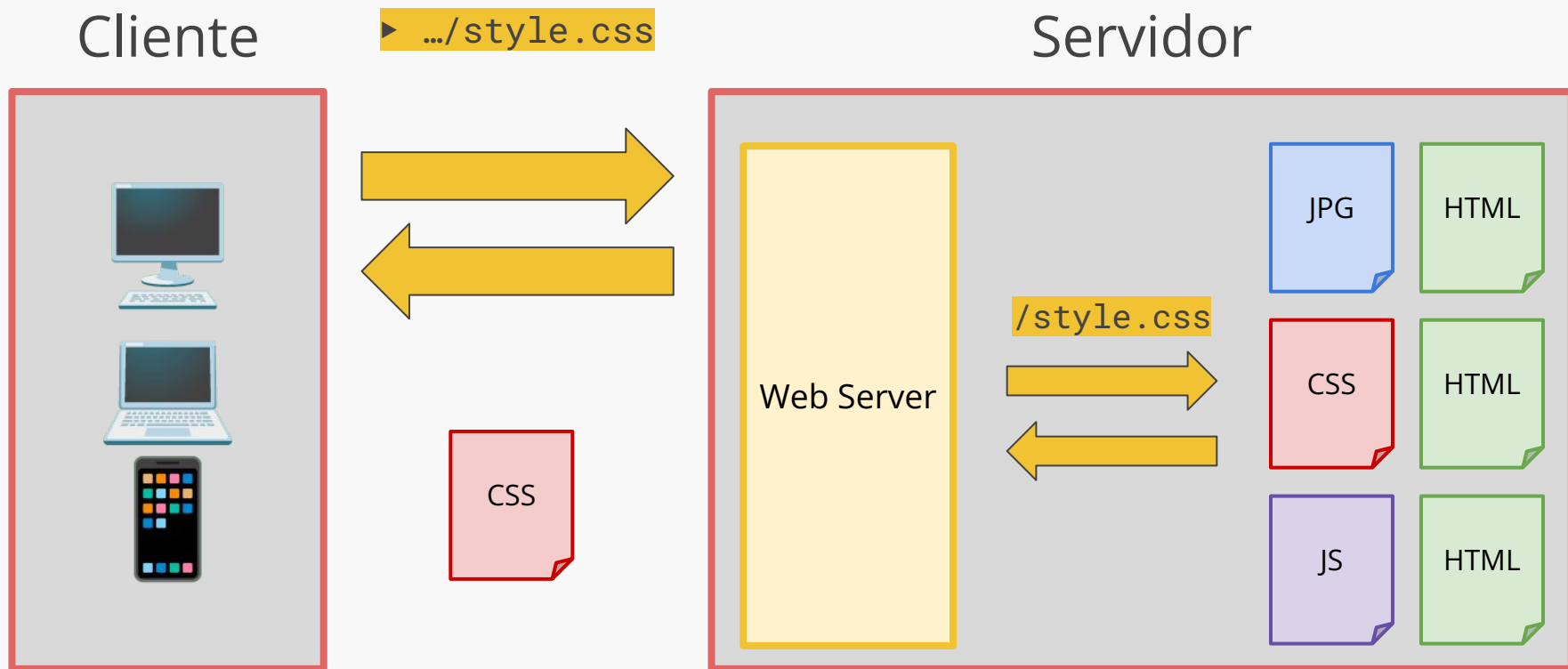
Más documentos que solo HTML



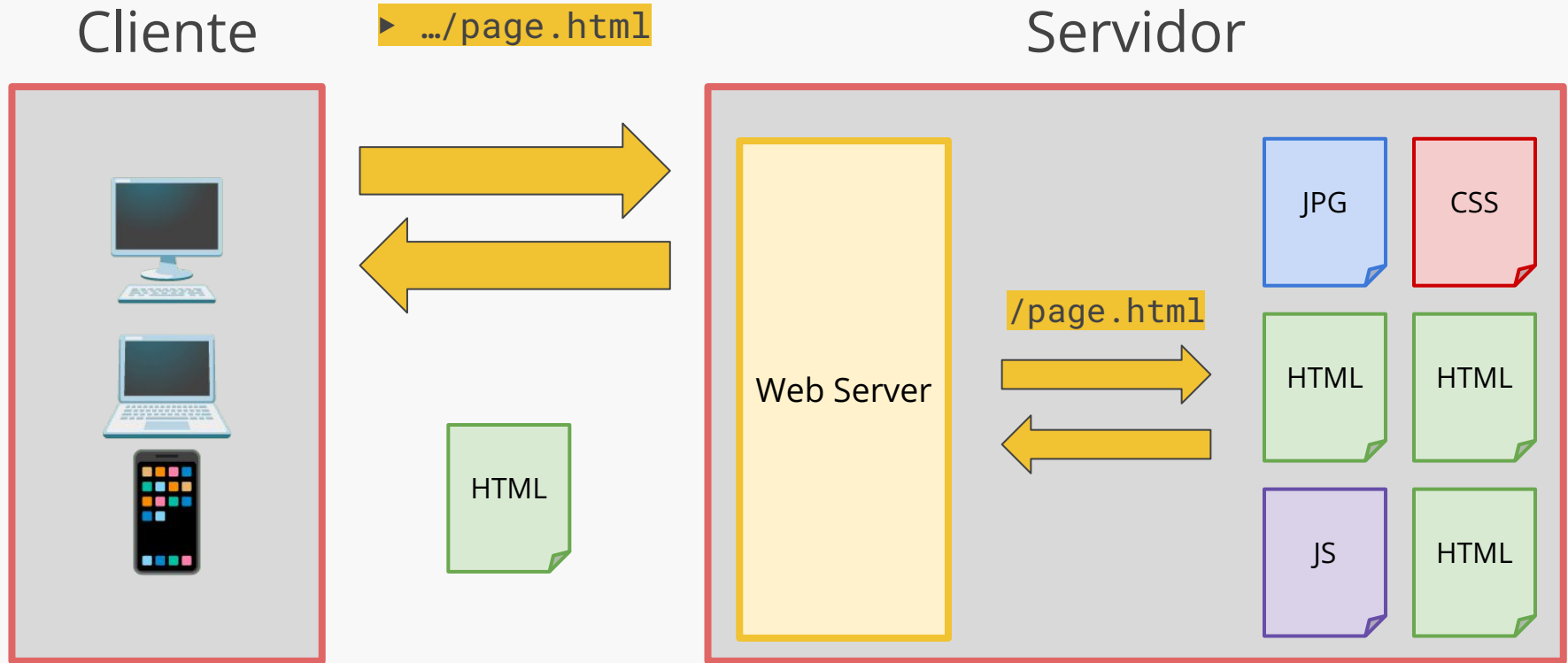
Aparece JavaScript en 1995



Aparece CSS en 1994-1996



Sitio web estático: un documento por página



Sitio web estático: un documento por página

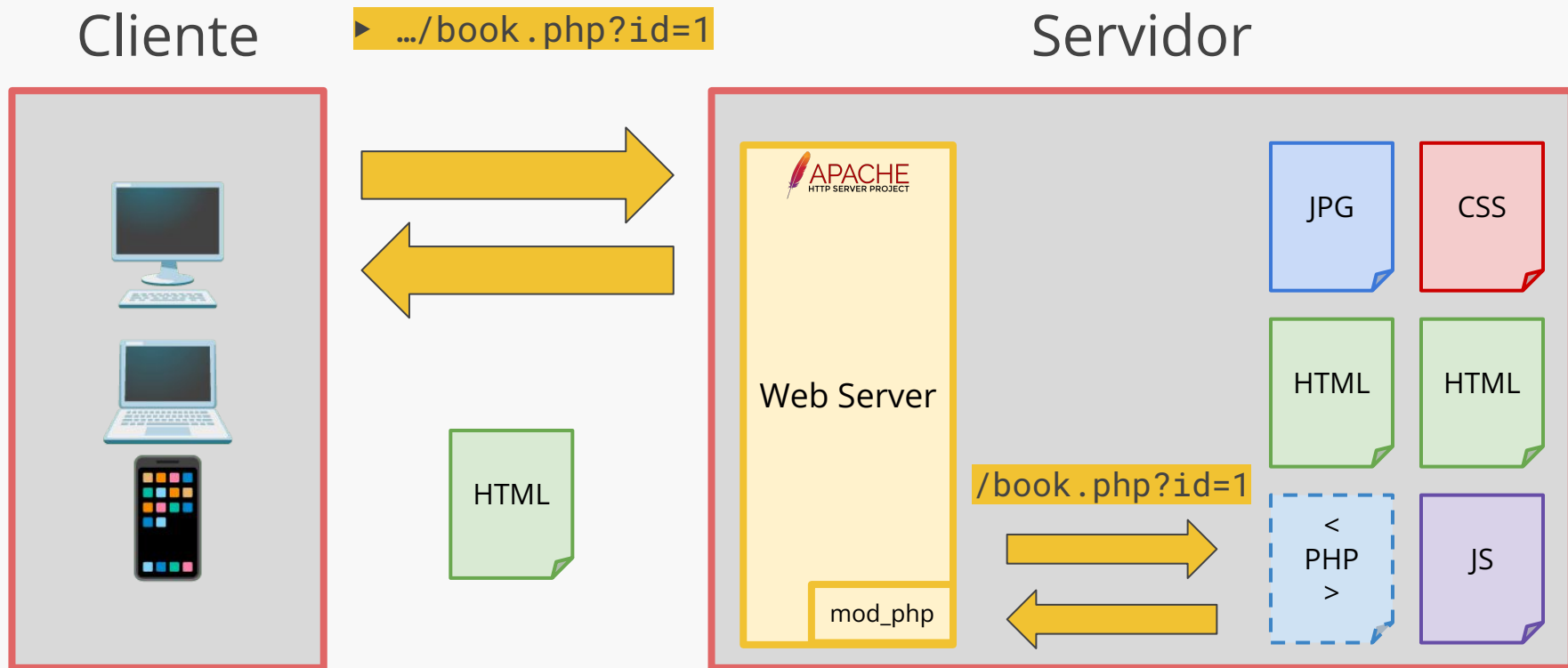
Librería con 20 libros ► 20 documentos HTML

...

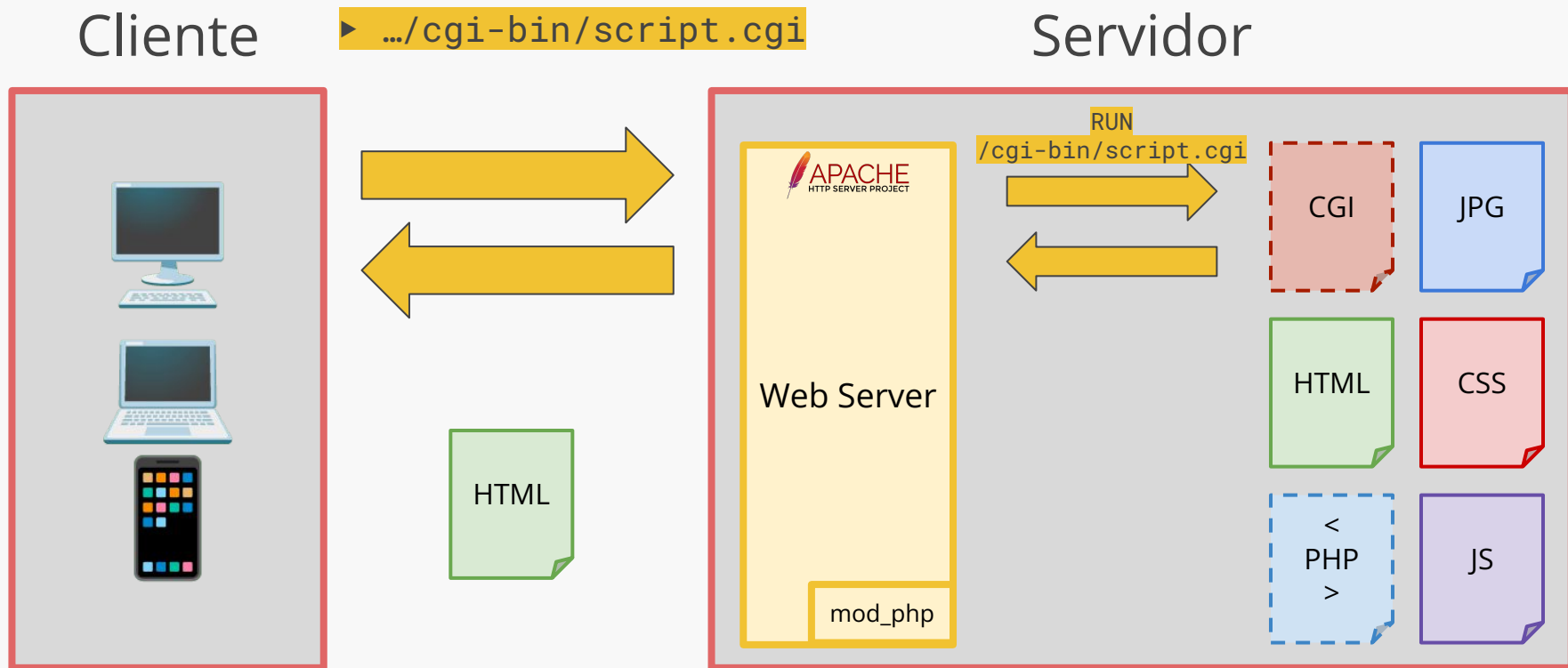
¿Y si quisiéramos tener **100 libros**?

¿Y si quisiéramos **cambiar la forma
en la que mostramos** los libros?

No solo entrega HTML, también ejecuta scripts



No solo entrega HTML, también ejecuta scripts



mod_php y CGI scripts

mod_php

Módulo de Apache HTTP Server que permite que el servidor web ejecute código PHP directamente

```
1 <!DOCTYPE html>
2 <html>
3     <head>
4         <title>PHP Test</title>
5     </head>
6     <body>
7         <?php echo '<p>Hello World</p>'; ?>
8     </body>
9 </html>
10
```

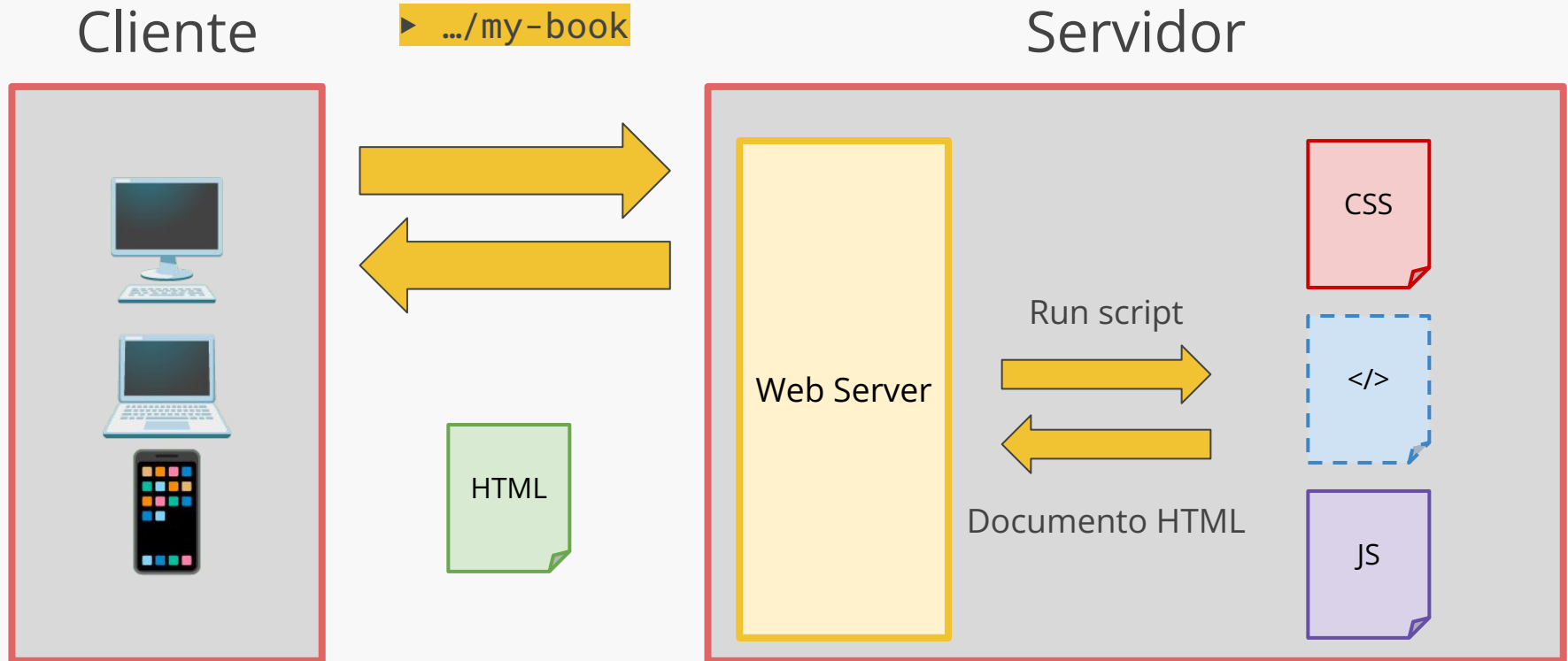
mod_php y CGI scripts

CGI script

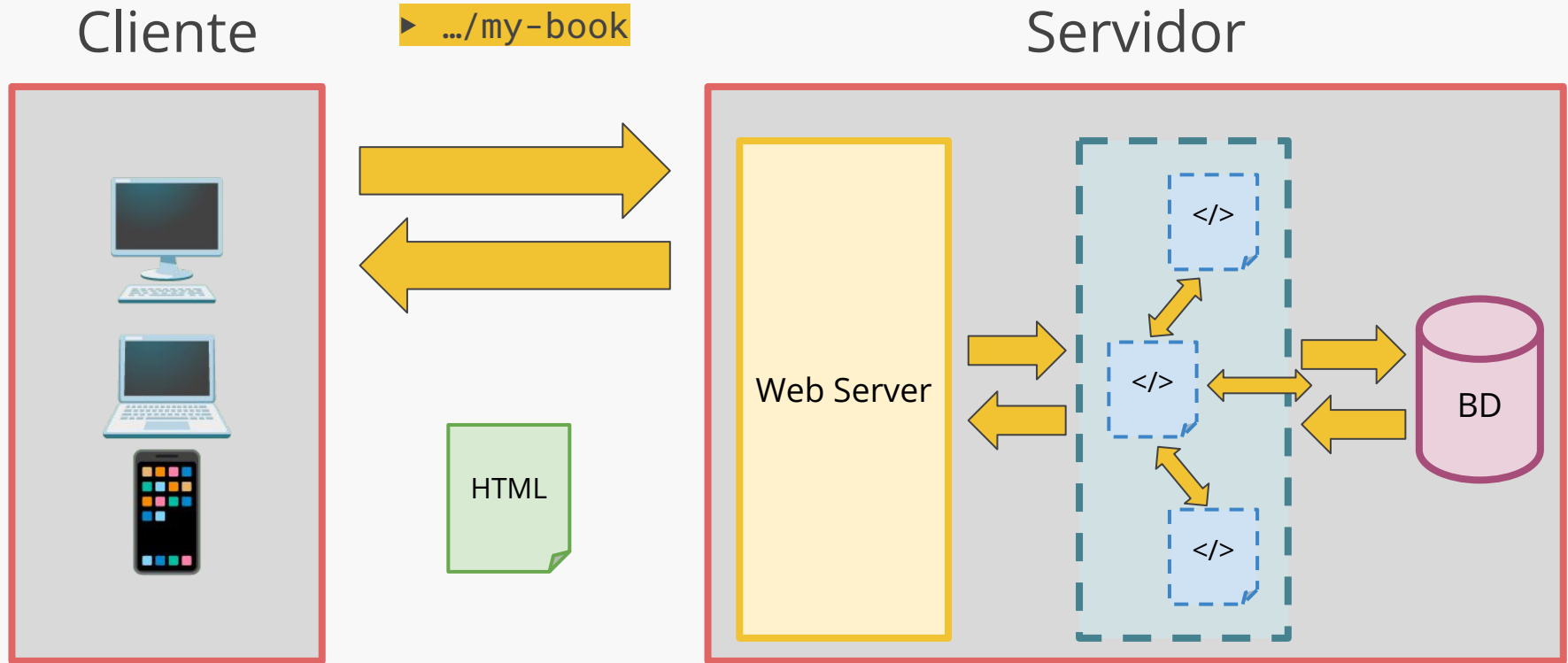
Programa escrito en un lenguaje como Perl, C o C++ que procesa datos en un servidor web. Mecanismo inicial usado en la interacción con BDs

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Content-type: text/html\n\n");
6     printf("<html>\n");
7     printf("<body>\n");
8     printf("<h1>Hello there!</h1>\n");
9     printf("</body>\n");
10    printf("</html>\n");
11    return 0;
12 }
```

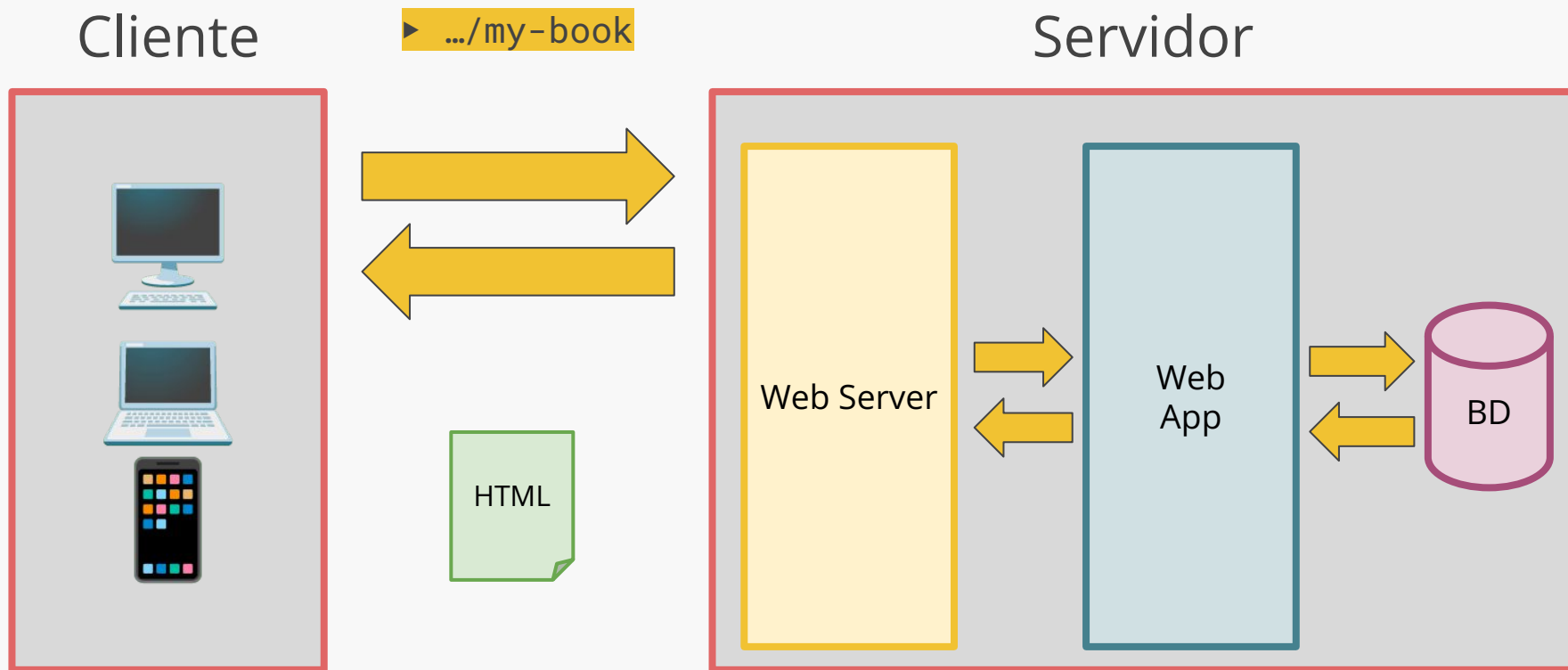
Sitio web dinámico: documento HTML generado



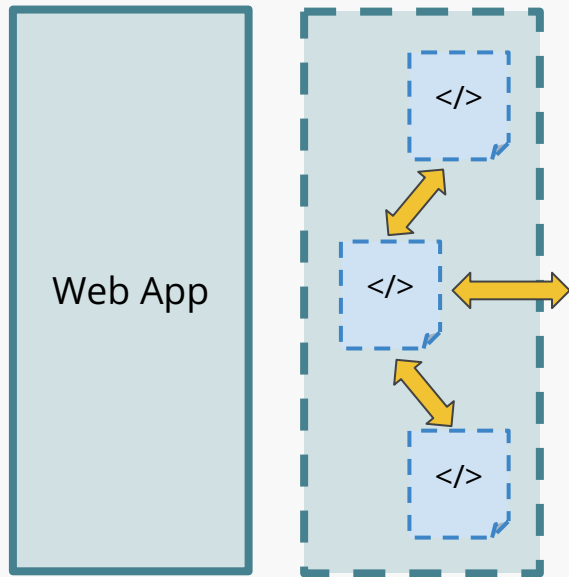
Complejidad crece (fines 90's - principios 2000)



Aparece la aplicación web



Aplicación web (*web app*)

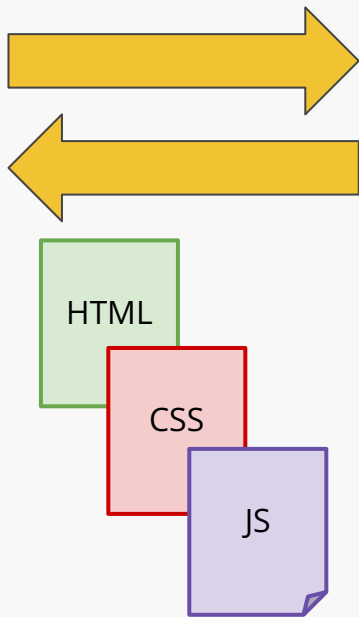


Software que realiza **acciones en el servidor** en base a una request HTTP

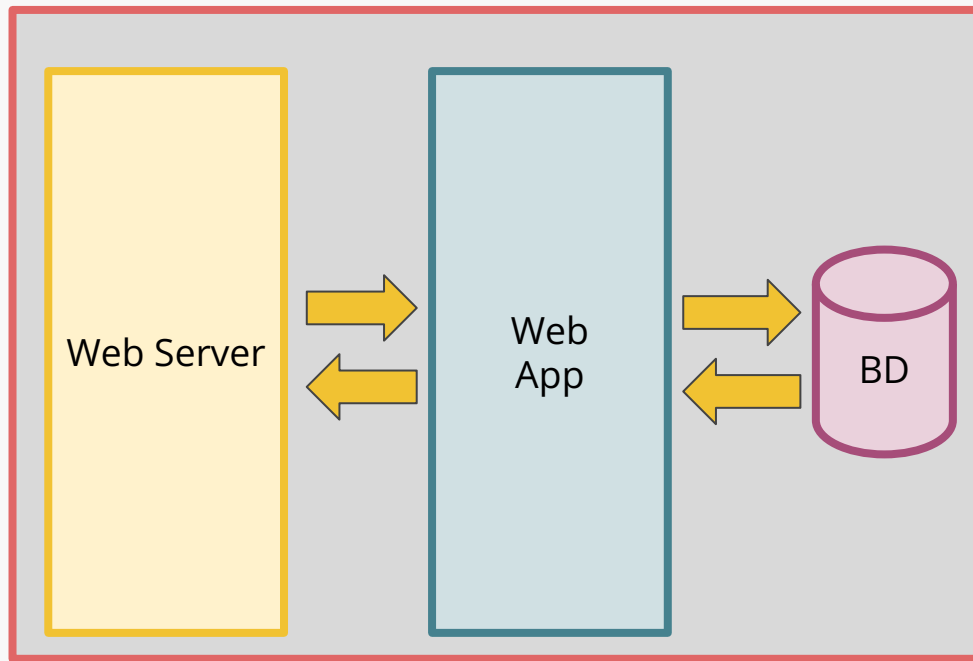
Construye de forma dinámica una respuesta, la cual se envía como una response HTTP

Aparece la aplicación web

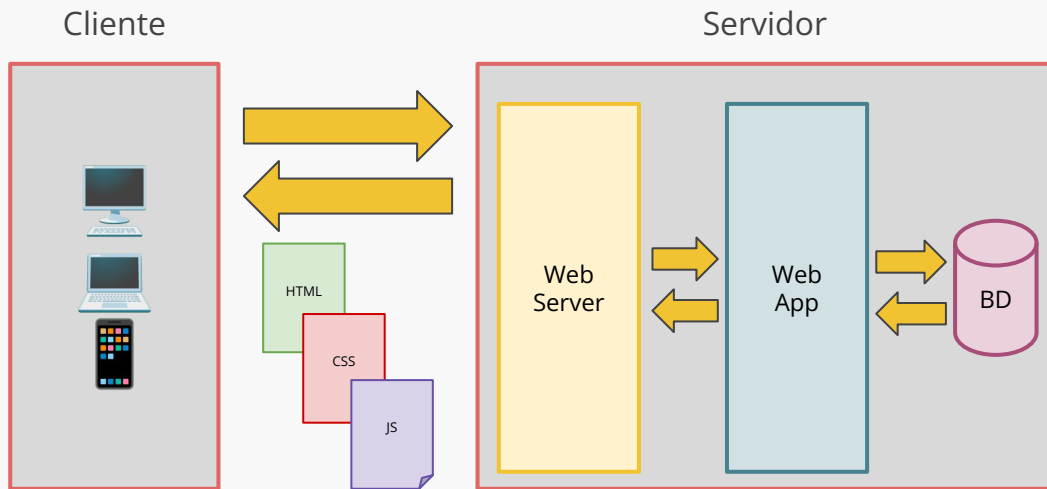
Cliente



Servidor



Coordinación cliente-servidor



1. Request recurso web
2. Servidor consulta DB
3. Servidor genera HTML
4. Cliente recibe e interpreta HTML
5. Documento HTML tiene referencia a CSS
6. Request al servidor del documento CSS
7. Servidor devuelve documento CSS
8. Cliente despliega página web con estructura y estilo
9. Documento HTML tiene referencia a script JS
10. Request al servidor del script JS
11. Servidor devuelve script JS
12. Cliente recibe y ejecuta script JS

Agenda

Cliente y servidor

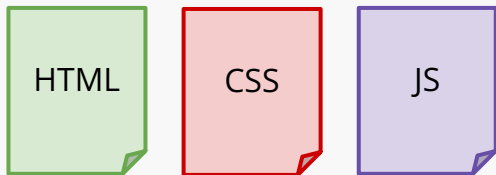


Frameworks web

Implementando MVC

En términos de tecnologías...

Client



Front-end

Cómo se ve
"Look & feel"

Server



Back-end

Cómo funciona
Lógica de negocio

En términos de tecnologías...

Front-end

Cómo se ve
“Look & feel”

- **Estructura (HTML)**
Contenido de una página web
- **Presentación (CSS)**
Aplicar *layout* y estilos visuales
- **Comportamiento (JS)**
Facilitar la interacción con el usuario

Back-end

Cómo funciona
Lógica de negocio

- Buscar y/o persistir información en una base de datos (BD)
- Manejar sesiones de usuario
- Controlar acceso a contenido
- Conexión con servicios (correo, transacciones, etc.)

Algunos lenguajes y *frameworks* web

Node (JS)

Koa, Express

Python

Django, Flask

Ruby

Rails, Sinatra

.NET

ASP.NET, Blazor

Java

Spring, JSF, Struts

PHP

Laravel, CodeIgniter,
Symfony

Breve historia de *web frameworks*

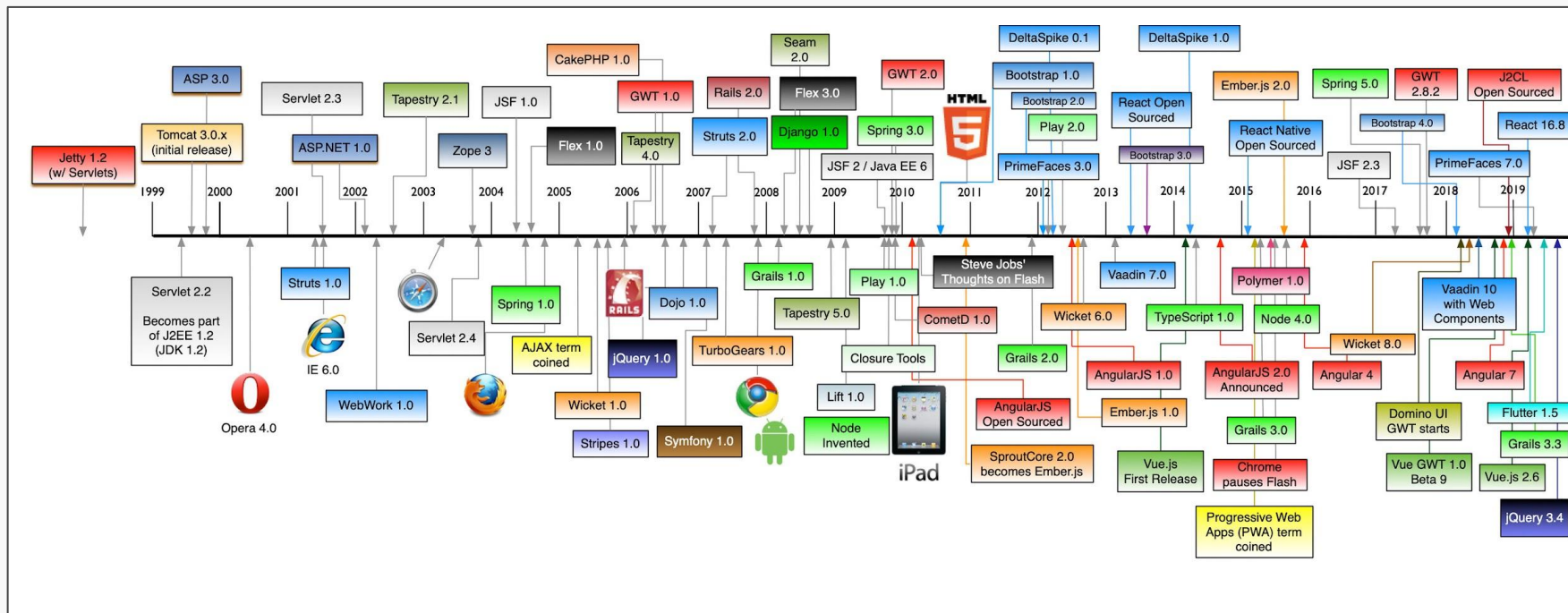


Figura: mraible/history-of-web-frameworks-timeline | [GitHub](https://github.com)

Express - Node.js web application framework:

Página principal de Express, un framework para aplicaciones web en JavaScript bastante popular

express

 search

Home

Getting started

Guide

API reference

Advanced topics

Resources

Support

Blog



English



Express

[5.1.0](#)

Fast, unopinionated,
minimalist web
framework for [Node.js](#)

```
$ npm install express --save
```

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```



Express@5.1.0: Now the Default on npm with LTS Timeline

Express 5.1.0 is now the default on npm, and we're introducing an official LTS schedule for the v4 and v5 release lines. [Check out our latest blog for more information.](#)

Express - Node.js web application framework:

Página principal de Express, un framework para aplicaciones web en JavaScript bastante popular

express

 search

Home

Getting started

Guide

API reference

Advanced topics

Resources

Support

Blog



English



Express
Fast, unopinionated,
minimalist web
framework

```
const express = require('express');  
const app = express();  
const port = 3000;
```

```
app.get('/', (req, res) => {  
  res.send('Hello World!');  
});
```

```
app.listen(port, () => {  
  console.log(`Example app listening on port ${port}`);  
});
```

`\${port}`)

\$ npm install express -



Express@5.1.0: Now the Default on npm with LTS Timeline

Express 5.1.0 is now the default on npm, and we're introducing an official LTS schedule for the v4 and v5 release lines. [Check out our latest blog for more information.](#)

Koa - next generation web framework for node.js:

Página principal de Koa, un framework web minimalista para Node.js



koa

next generation web framework for node.js

Koa - next generation web framework for node.js:

Página principal de Koa, un framework web minimalista para Node.js



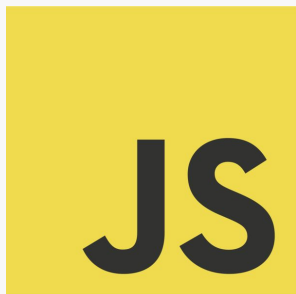
```
const Koa = require('koa');
const app = new Koa();

app.use(async ctx => {
  ctx.body = 'Hello World';
});

app.listen(3000);
```

En el caso del proyecto...

Lenguaje



En el caso del proyecto...

Lenguaje



Back-end
(servidor)

express
koa

En el caso del proyecto...

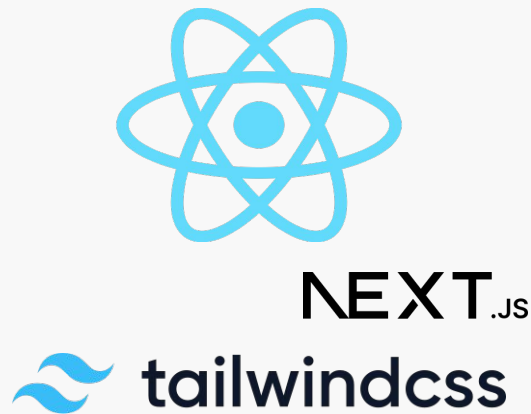
Lenguaje



Back-end
(servidor)

express
koa

Front-end
(cliente)



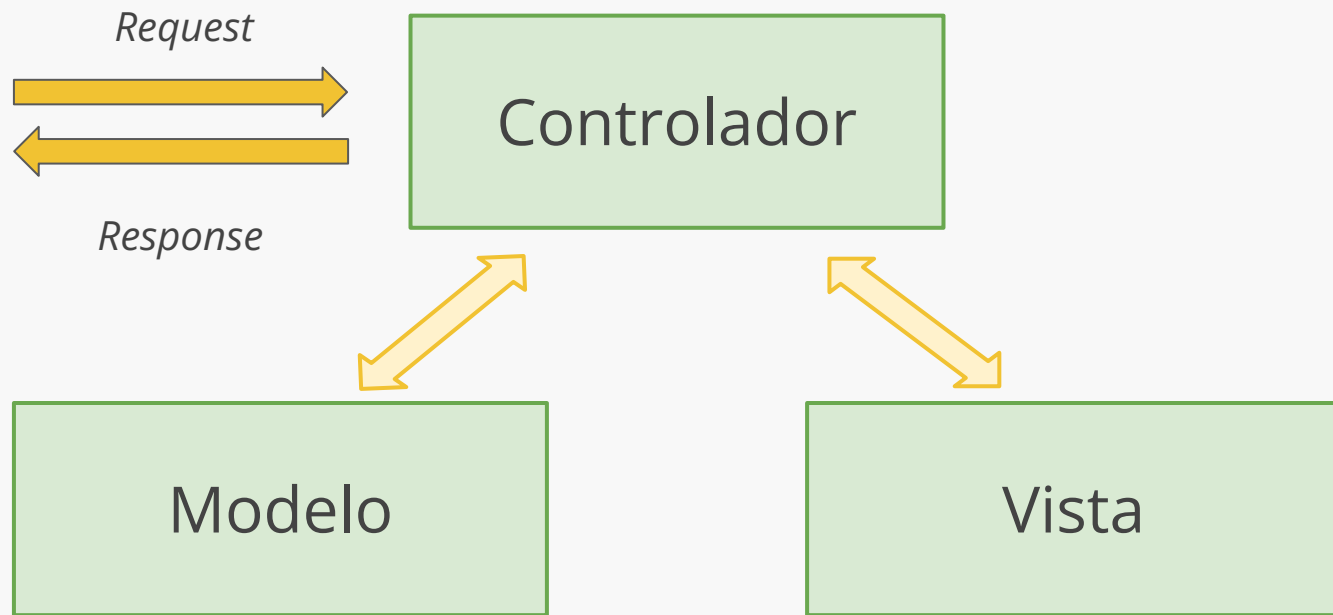
Agenda

Cliente y servidor

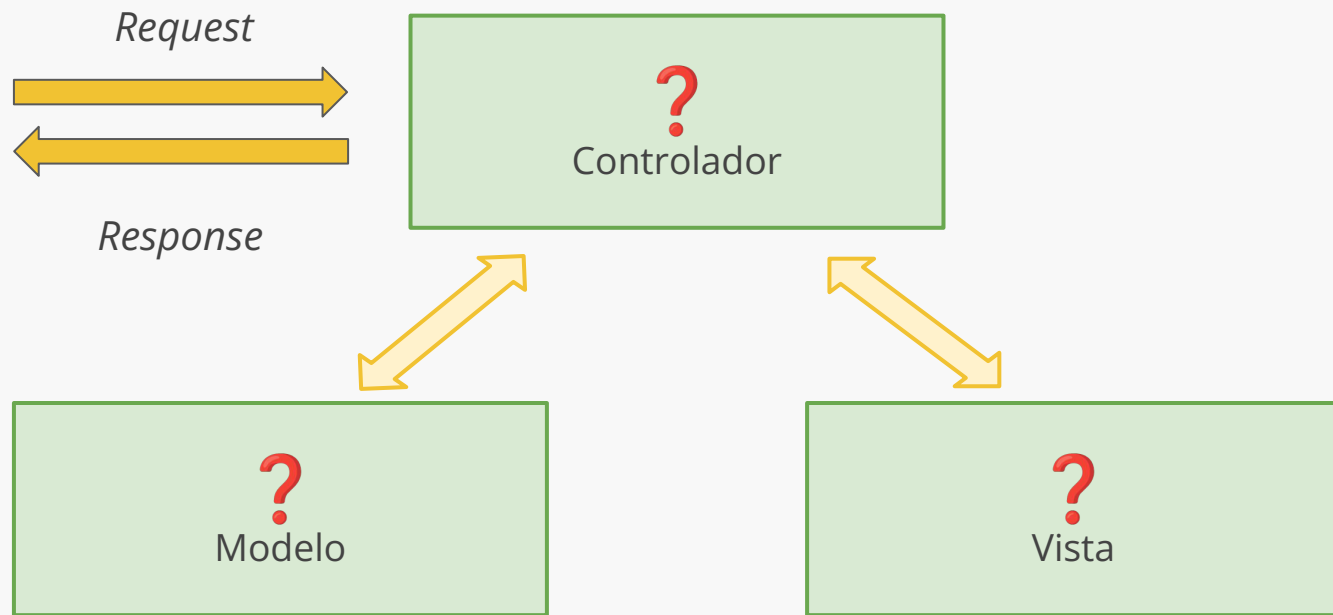
Frameworks web

✨ **Implementando MVC**

MVC: Modelo - Vista - Controlador



¿Cómo se ve el proyecto del curso?



Agenda

Cliente y servidor

Frameworks web

Implementando MVC

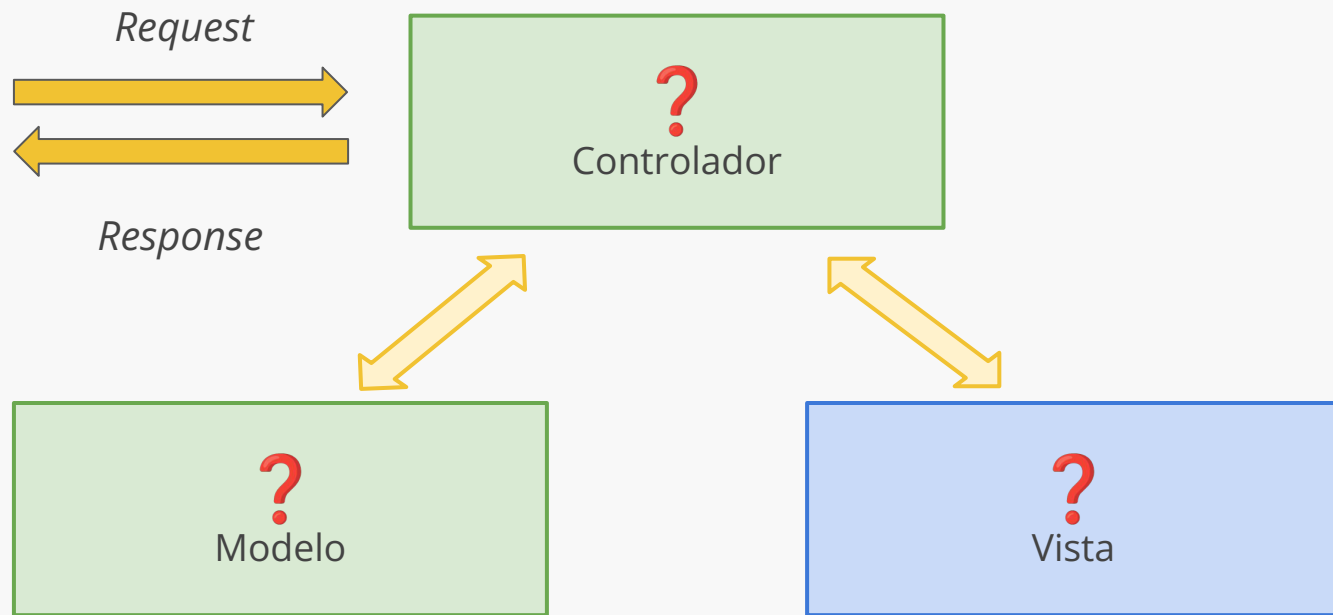


Vista

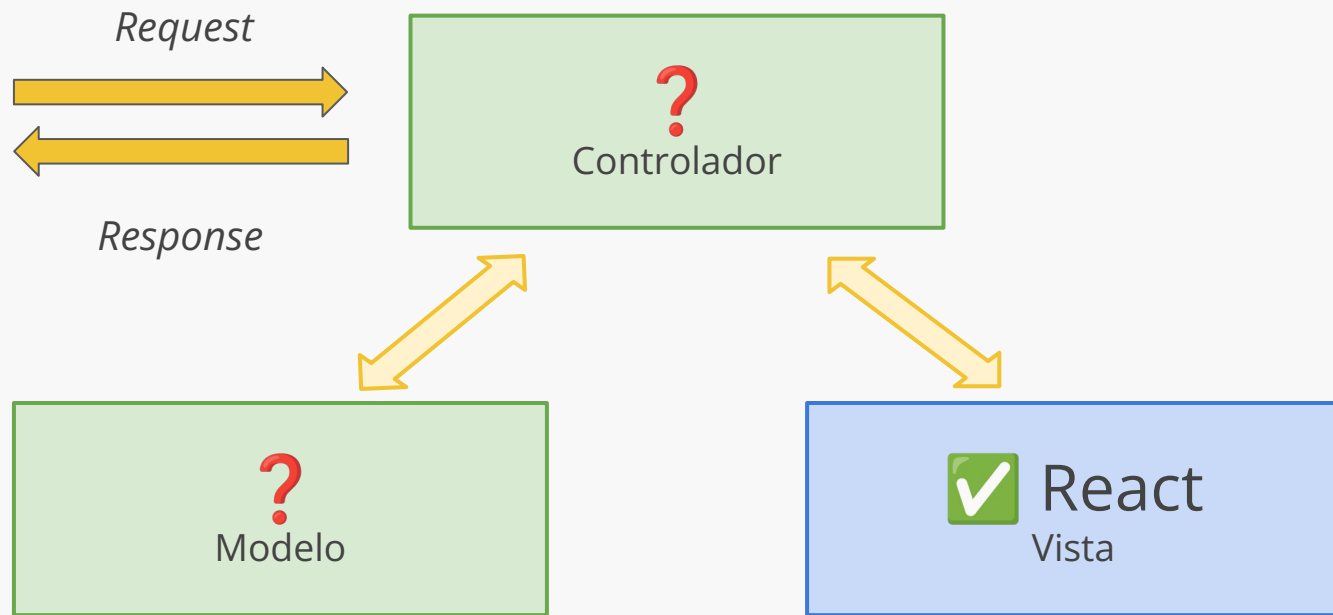
Controlador

Modelo

¿Cómo se ve el proyecto del curso?



¿Cómo se ve el proyecto del curso?



CSR: Client Side Rendering

Útil cuando se requiere mucha interacción del cliente y solo tenemos que actualizar los datos

Sin embargo, la carga inicial puede ser lenta (se envía mucho código en JS) y la página inicialmente está en blanco

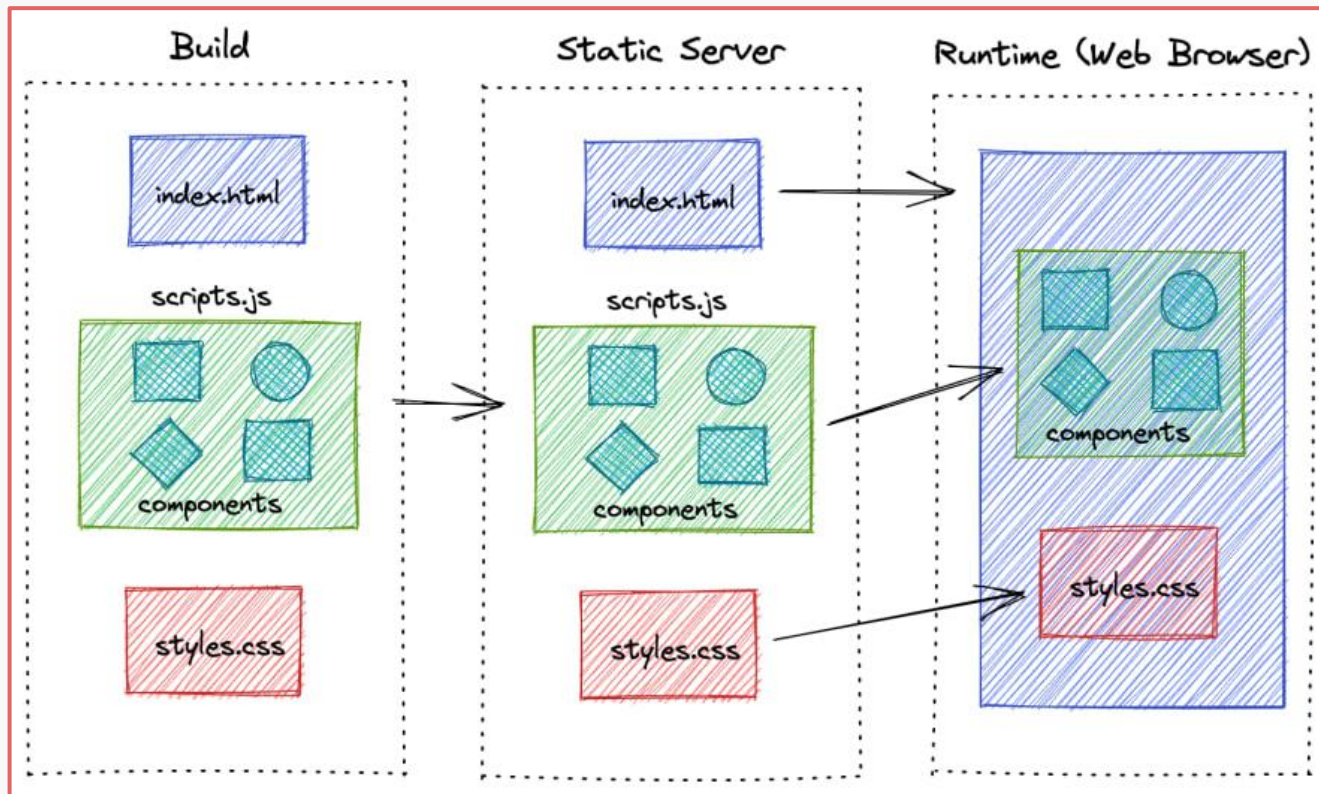


Figura: [Visual Explanation and Comparison of CSR, SSR, SSG and ISR - DEV Community](#)

CSR: Client Side Rendering

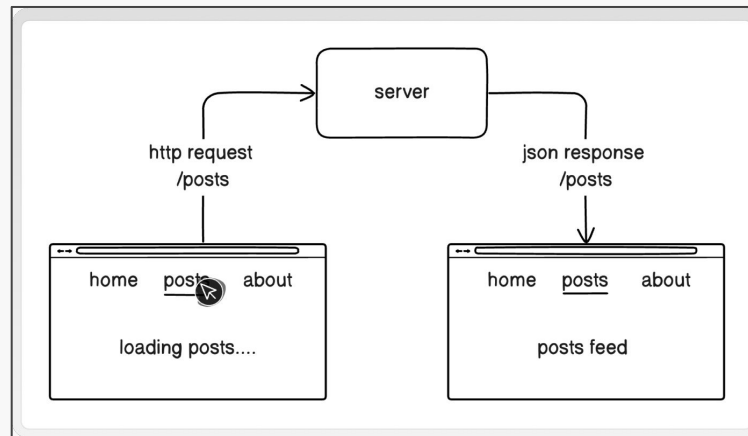
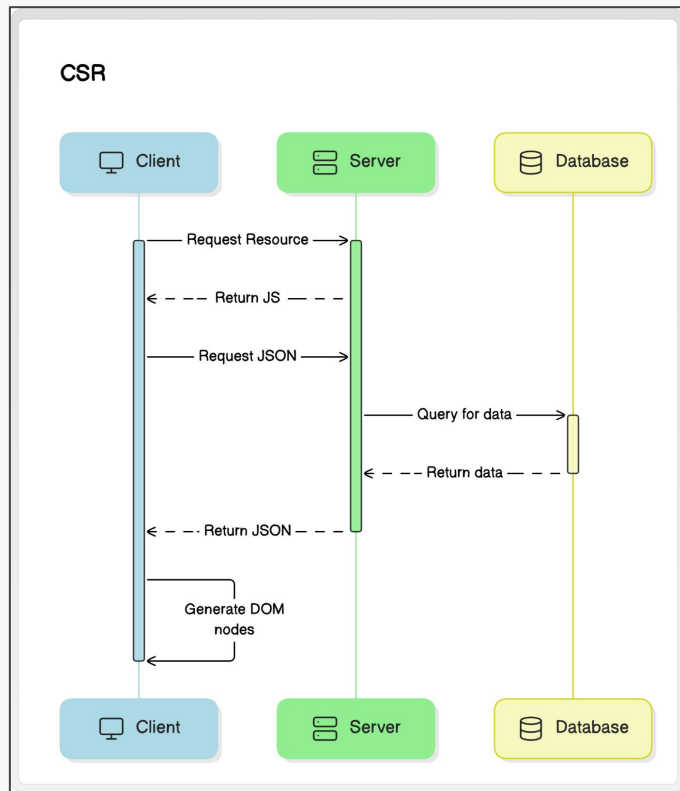
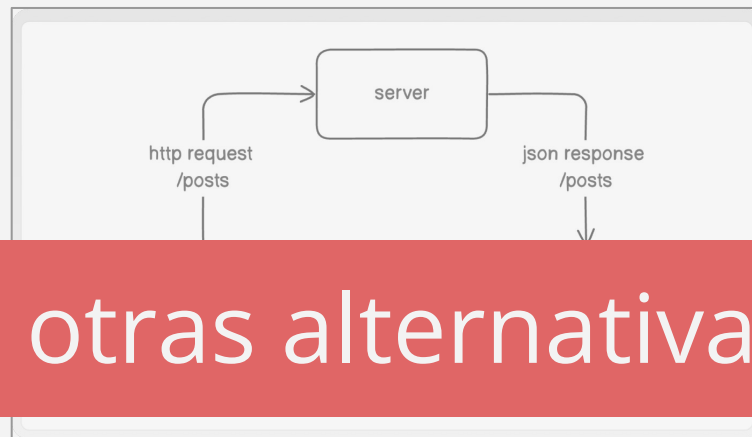
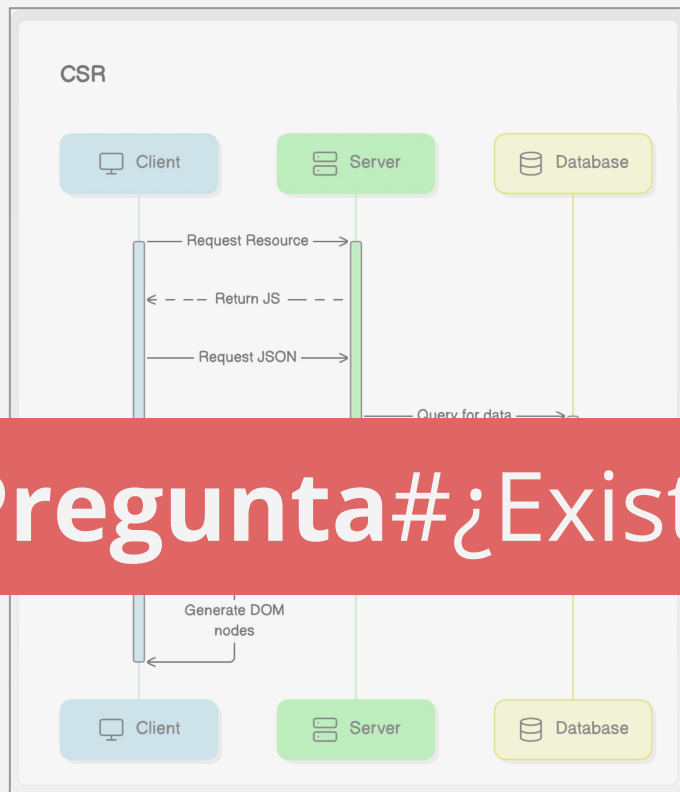


Figura: [Server Side Rendering for Web Apps: The Tradeoffs You're Making - Decision Node](#)

CSR: Client Side Rendering

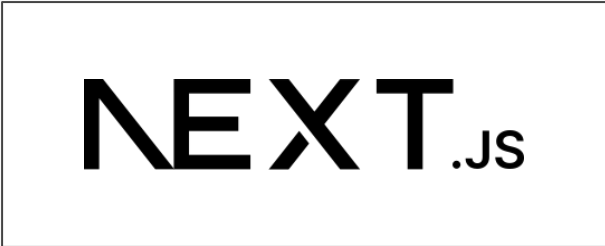


Pregunta#¿Existen otras alternativas?

Next.js

A diferencia de React, que es una librería, se dice que [Next.js](#) es un framework de React. Esto significa que incluye herramientas y configuración necesarias para React, y provee adicionalmente estructura, *features* y optimizaciones

Next.js fue creado por Vercel, una compañía que (en simple) permite publicar aplicaciones web basadas en front-end. El proyecto fue hecho público en su propio [repositorio en GitHub](#) en 2016

The logo for Next.js, featuring the word "NEXT" in a bold, black, sans-serif font, followed by ".JS" in a smaller, black, sans-serif font. The logo is centered within a thin black rectangular border.

NEXT.js

Next.js

A diferencia de React, que es una librería, se dice que [Next.js](#) es un framework de React. Esto significa que incluye herramientas y configuración necesarias para React, y provee adicionalmente estructura, *features* y optimizaciones



Al ser un framework sobre React, es necesario que conozcan bien React antes de trabajar con Next.js

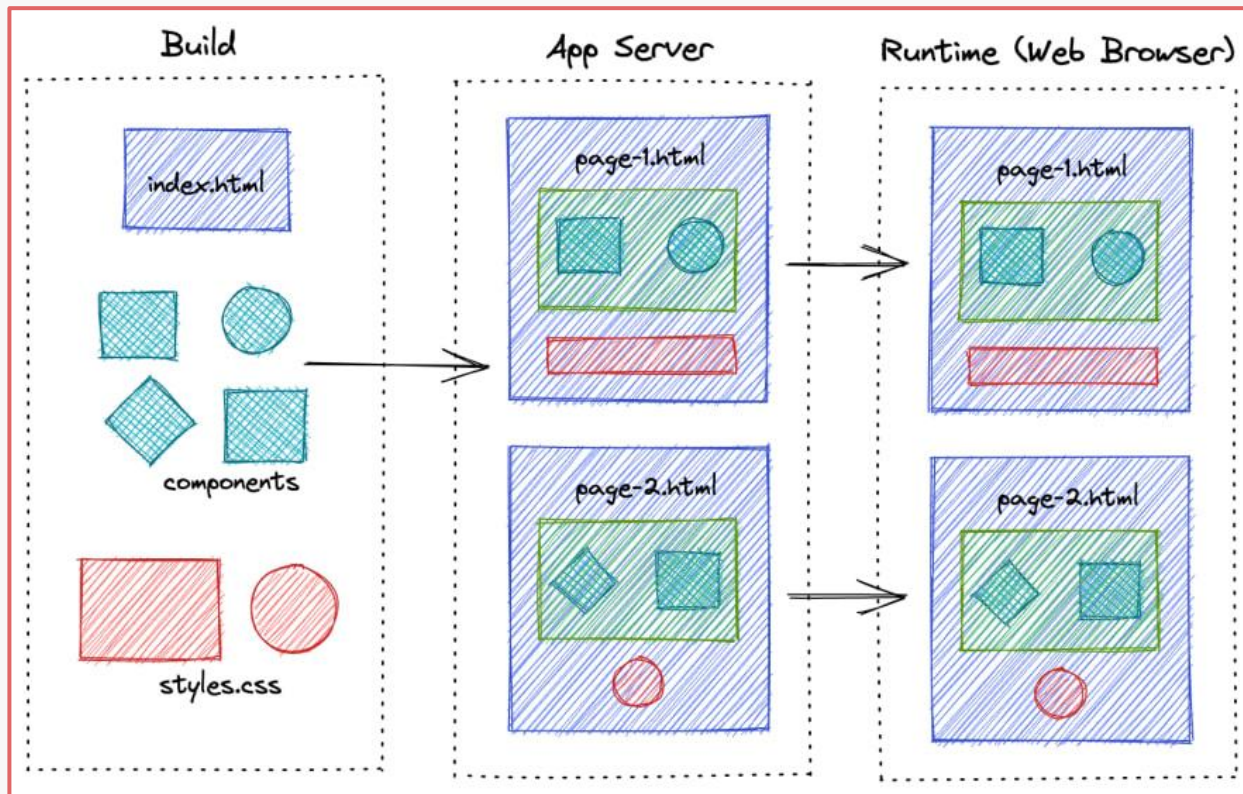
(consideren esto antes de decidir lanzarse con Next.js)

El creador de Next.js, [Shaykhov](#), lanzó su propio [repositorio en GitHub](#) en 2016

SSR: Server Side Rendering

Útil cuando las páginas tienen que mostrar datos que cambian frecuentemente y tienen que estar actualizados en cada *request* (por ejemplo, perfiles de usuario, *dashboards* en tiempo real)

Sin embargo, dado que el renderizado ocurre en cada *request* y en el servidor, el servidor se lleva una mayor carga



SSR: Server Side Rendering

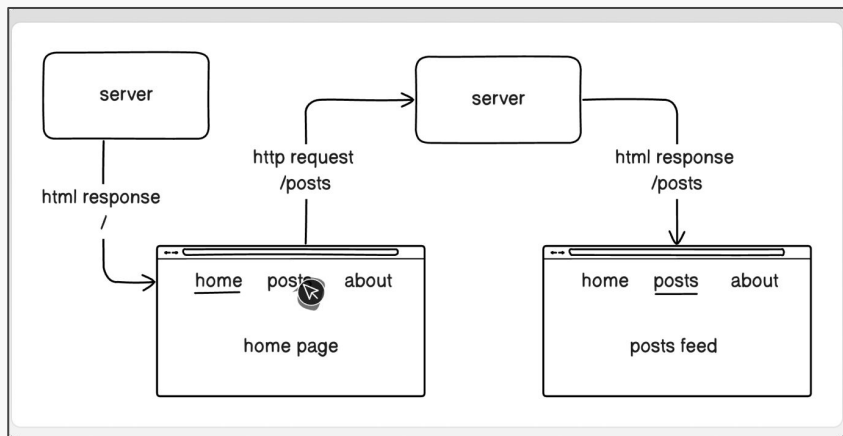
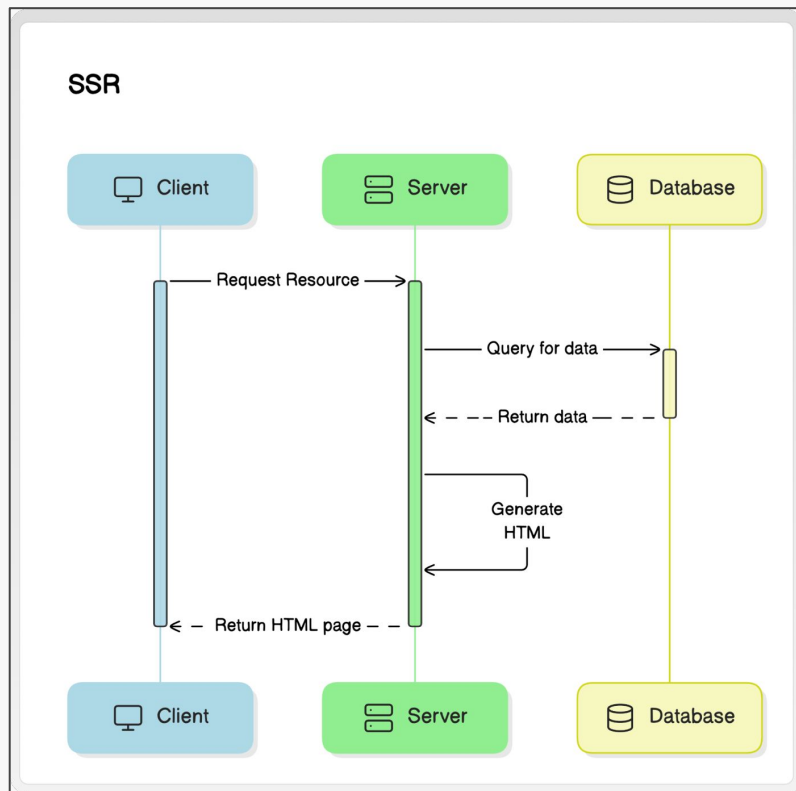


Figura: [Server Side Rendering for Web Apps: The Tradeoffs You're Making - Decision Node](#)

SSG: Static Site Generation

Con SSG el contenido está disponible de inmediato, es la alternativa más rápida porque el HTML ya está listo, por lo que es la mejor alternativa al considerar SEO (optimización para motores de búsqueda)

El uso de SSG es ideal para sitios que se basan en contenido estático, tales como blogs, portafolios, documentación, etc.

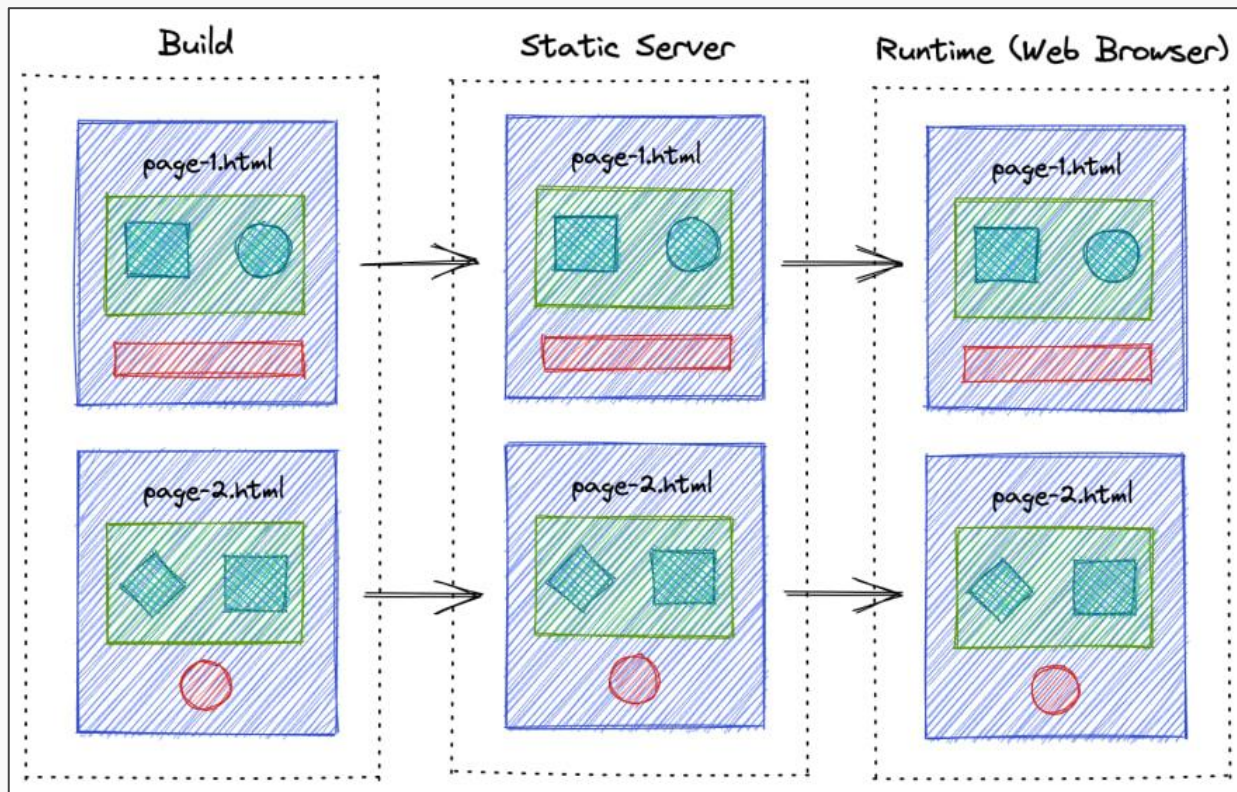


Figura: [Visual Explanation and Comparison of CSR, SSR, SSG and ISR - DEV Community](#)

SSG: Static Site Generation

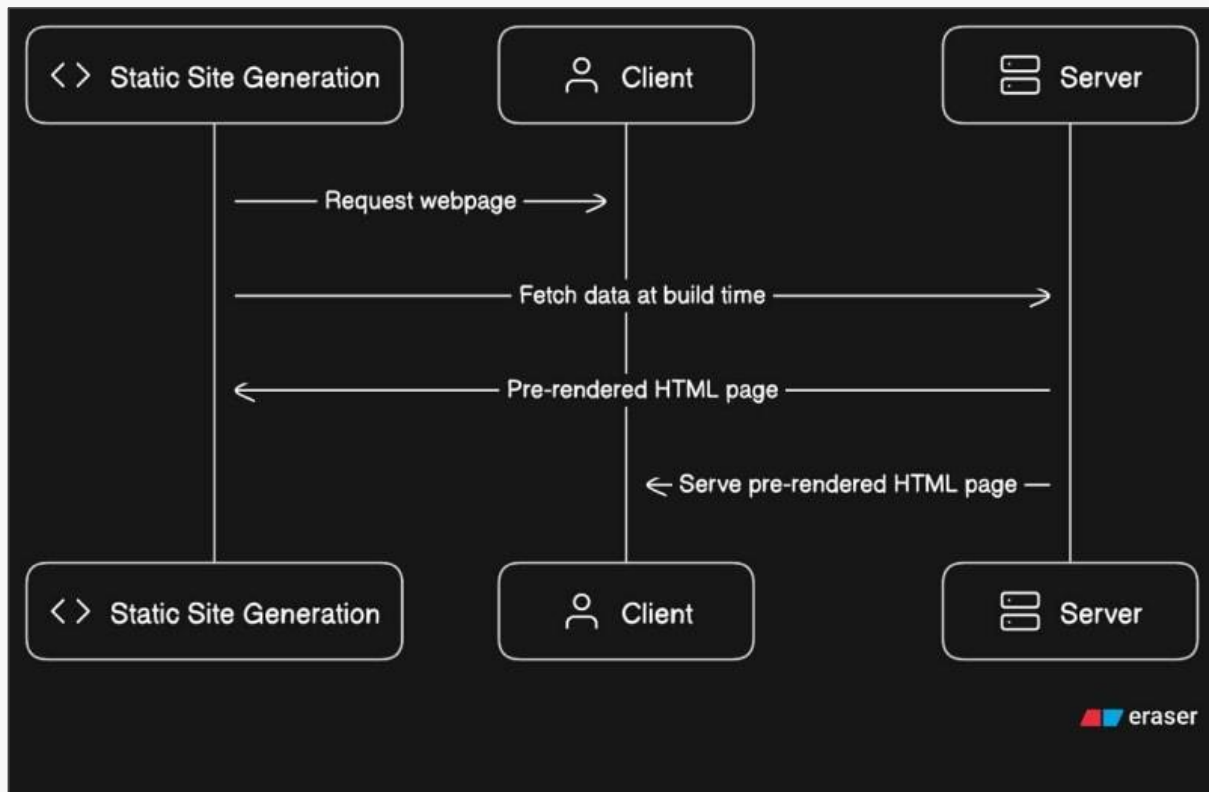


Figura: [Exploring the Differences between SSR, SSG, CSR and ISR. Which is the Best Approach? \(with diagrams\) - DEV Community](#)

ISR: Incremental Static Regeneration

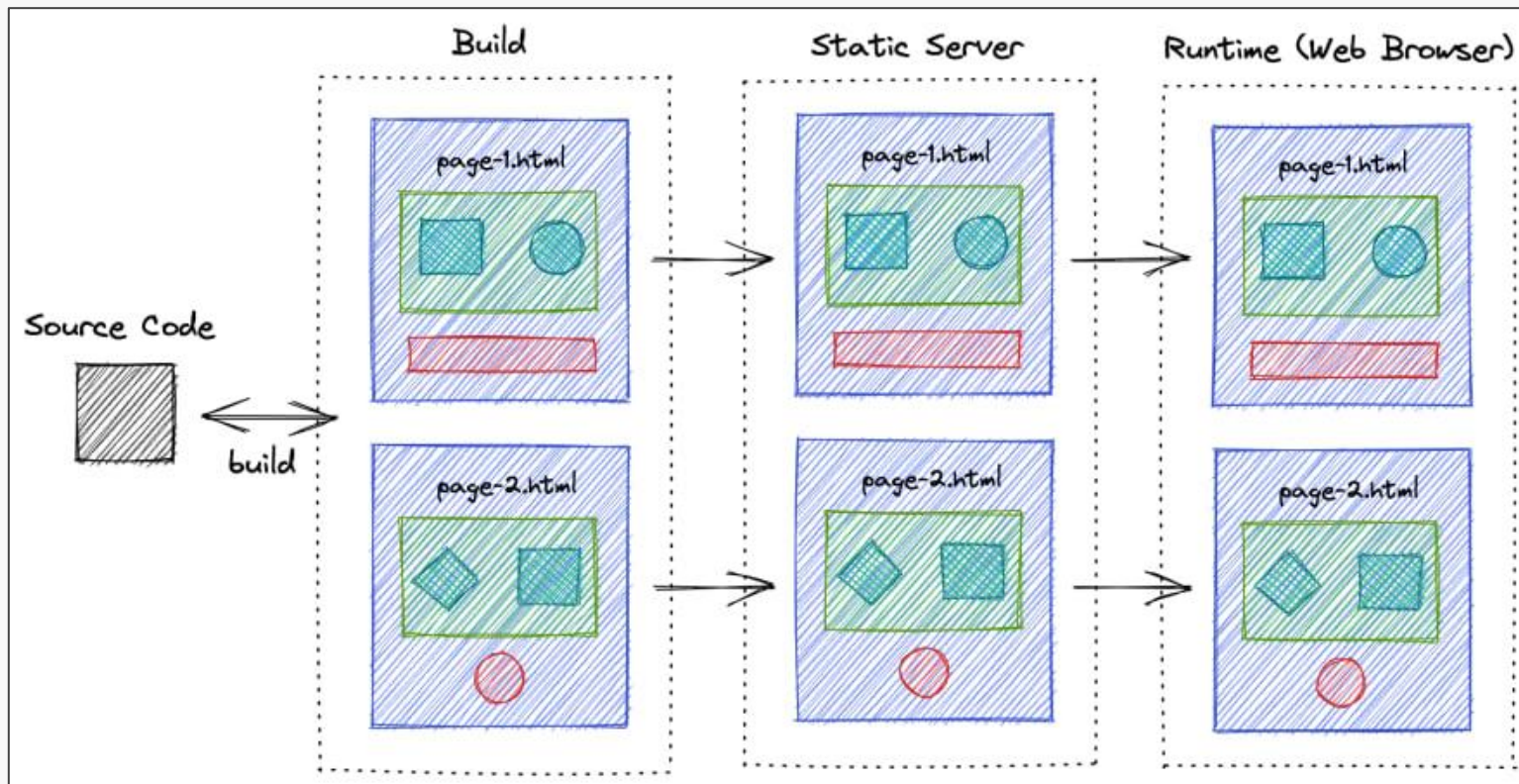


Figura: [Visual Explanation and Comparison of CSR, SSR, SSG and ISR - DEV Community](#)

ISR: Incremental Static Regeneration

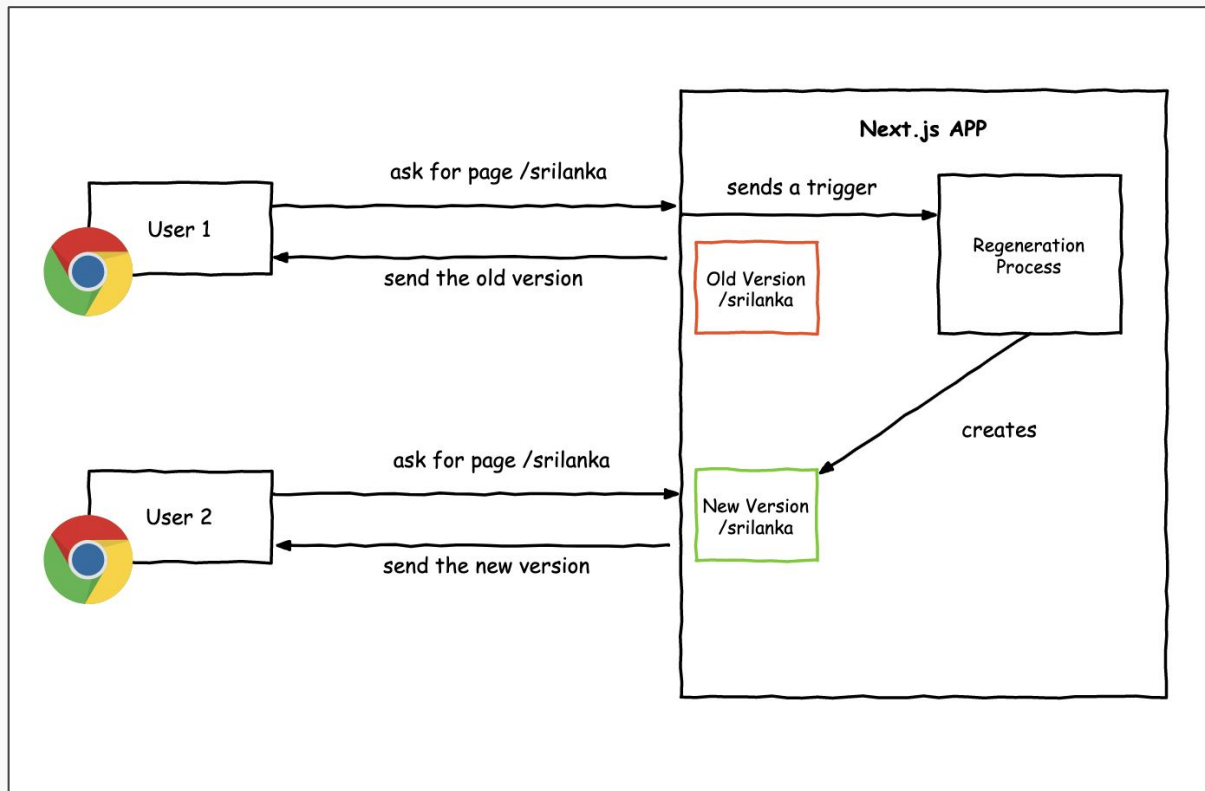


Figura: [What is Next.js Incremental Static \(Re\)Generation?](#)

 **Using Pages Router**
Features available in /pages **Using Latest Version**
15.2.4

Getting Started

[Installation](#)[Project Structure](#)

Building Your Application

[Routing](#)[Rendering](#)[Server-side Rendering \(SSR\)](#)[Static Site Generation \(SSG\)](#)[Automatic Static Optimization](#)[Client-side Rendering \(CSR\)](#)[Edge and Node.js Runtimes](#)[Data Fetching](#)[Pages Router](#) > [Building Your Application](#) > [Rendering](#)

Rendering

By default, Next.js **pre-renders** every page. This means that Next.js generates HTML for each page in advance, instead of having it all done by client-side JavaScript. Pre-rendering can result in better performance and SEO.

Each generated HTML is associated with minimal JavaScript code necessary for that page. When a page is loaded by the browser, its JavaScript code runs and makes the page fully interactive (this process is called [hydration ↗](#) in React).

Pre-rendering

Next.js has two forms of pre-rendering: **Static Generation** and **Server-side Rendering**. The difference is in **when** it generates the HTML for a page.

- **Static Generation:** The HTML is generated at **build time** and will be reused on each request.
- **Server-side Rendering:** The HTML is generated on **each request**.

On this page

[Pre-rendering](#)[Edit this page on GitHub ↗](#)[Managed Next.js \(Vercel\) ↗](#)

Agenda

Cliente y servidor

Frameworks web

Implementando MVC

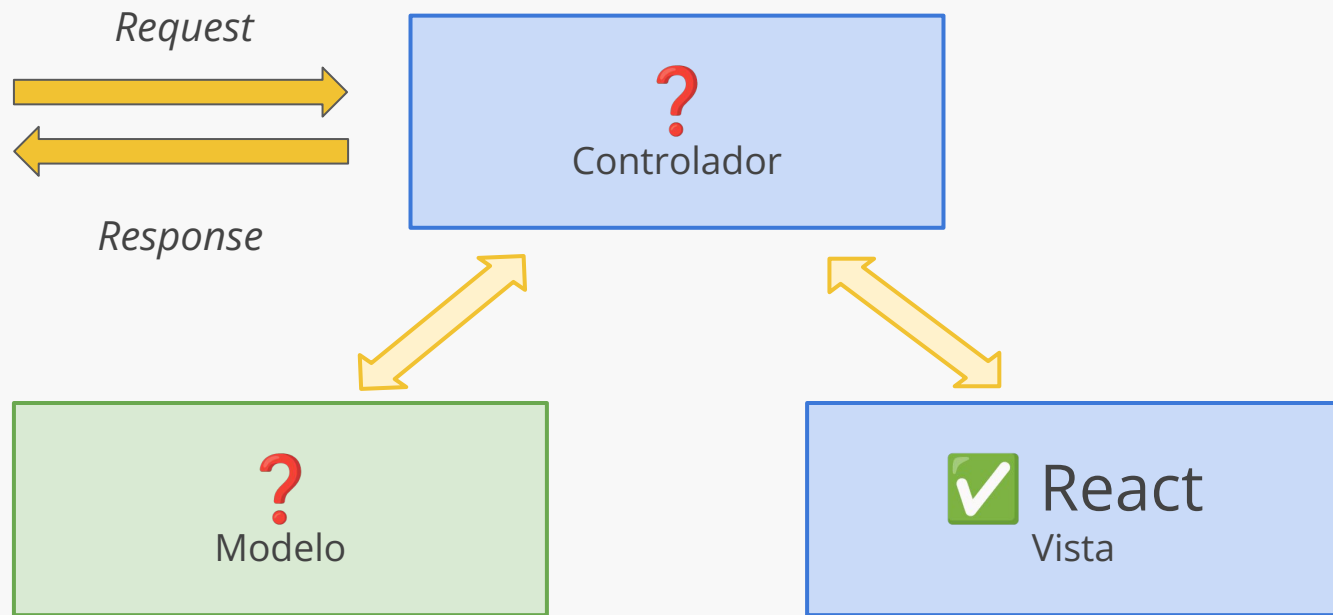
Vista



Controlador

Modelo

¿Cómo se ve el proyecto del curso?



Express

Es un framework web de Node.js creado pensando en ser una implementación minimalista y flexible que permita construir servidores HTTP de alta performance

Express fue creado en 2010 (tal como indica su [repositorio en GitHub](#))

The word "express" is displayed in a thin, lowercase, sans-serif font, centered within a rectangular border.

express

¿Cómo se ve?

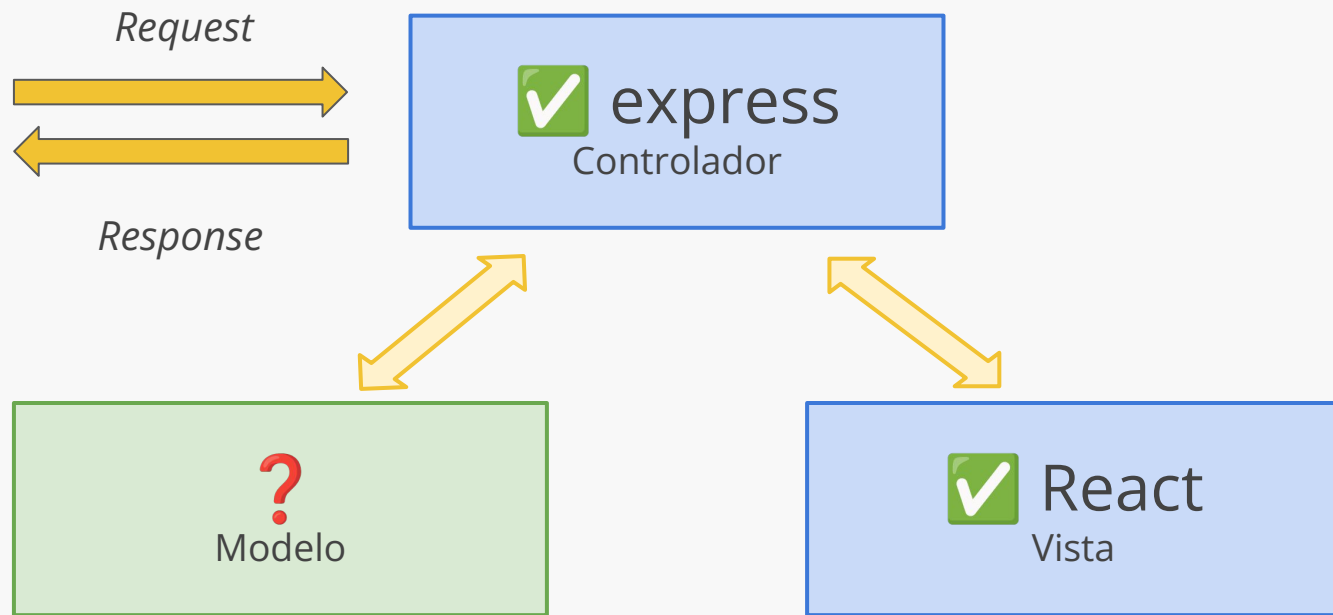
```
const express = require('express');  
const app = express();
```

```
app.get('/', (req, res) => {  
  res.send('Hello World!');  
});
```

```
app.listen(3000);
```

app.METHOD(PATH, HANDLER)

¿Cómo se ve el proyecto del curso?



Koa

Es un framework web de Node.js creado por el equipo de Express y construido para entregar una versión **minimalista y más moderna** que Express

[Koa](#) fue ideado para aprovechar lo que en su momento (2013) eran nuevas implementaciones en el manejo de la **asincronía en Node.js**, en particular, *async/await*, entregando un código de uso mucho más limpio

The Koa logo, featuring the word "koa" in a lowercase, serif font, enclosed within a thin black rectangular border.

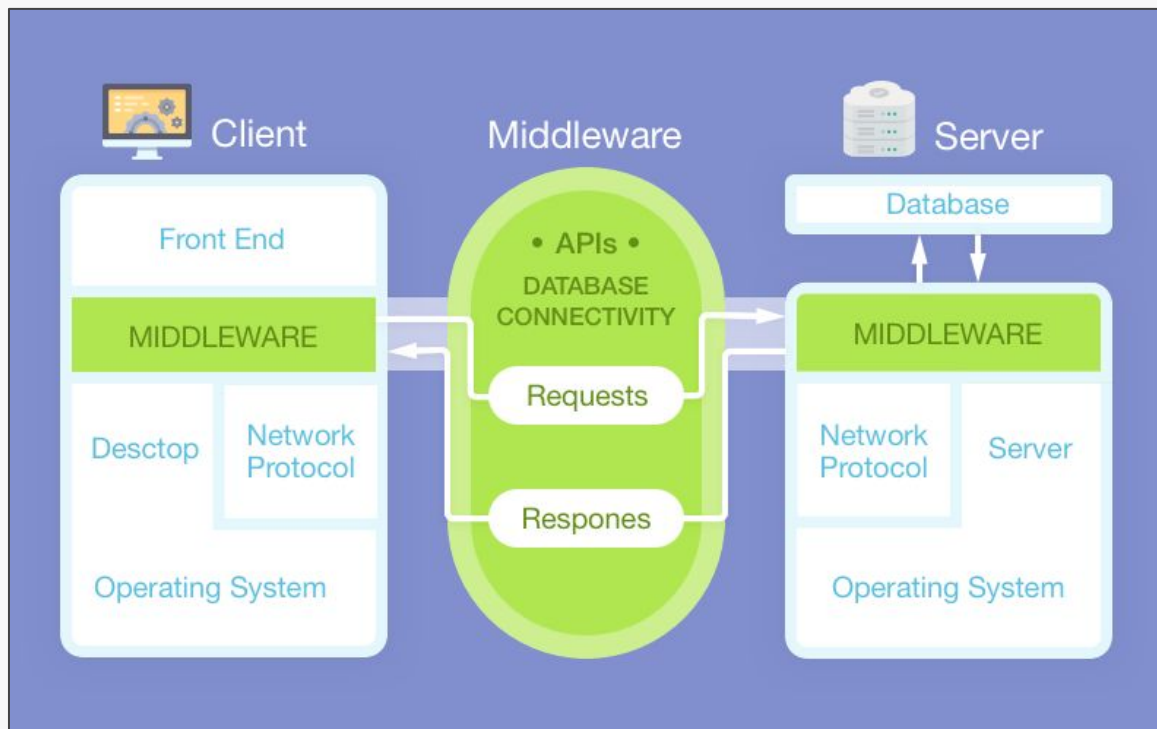
¿Cómo se ve?

```
const Koa = require('koa');  
const app = new Koa();  
  
app.use(async ctx => {  
  ctx.body = 'Hello World';  
});  
  
app.listen(3000);
```

Concepto de *middleware*

Pieza de software entre cliente y servidor, que interactúa con las respuestas y solicitudes entre ambas partes.

Algunas tareas típicas para un *middleware* son autenticación, validación, almacenar transacciones, etc.



¿Cómo se ve en Express?

```
const express = require('express');
const app = express();

const myLogger = function (req, res, next) {
  console.log('LOGGED');
  next();
};

app.use(myLogger);

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.listen(3000);
```

¿Cómo se ve en Koa?

```
const Koa = require('koa');
const app = new Koa();

// x-response-time
app.use(async (ctx, next) => {
  const start = Date.now();
  await next();
  const ms = Date.now() - start;
  ctx.set('X-Response-Time', `${ms}ms`);
});

// response
app.use(async ctx => {
  ctx.body = 'Hello World';
});

app.listen(3000);
```

¿Cómo se ve en Koa?

```
const Koa = require('koa');
const app = new Koa();

// x-response-time
app.use(async (ctx, next) => {
  const start = Date.now();
  await next();
  const ms = Date.now() - start;
  ctx.set('X-Response-Time', `${ms}ms`);
```

¿Y cómo manejamos rutas?

```
  ctx.body = 'Hello World';
});

app.listen(3000);
```

¿Qué es koa-router?

Simplemente es un *middleware* para Koa, que funciona como *router*, es decir, definir comportamiento para distintas rutas:

- Permite definir *callbacks* para cada método HTTP
- Permite encadenar/anidar *routers*
- Response a OPTIONS (método HTTP OPTIONS)
- Soporta *async/await*

Es pieza clave para desarrollar APIs RESTful con Koa (al igual que durante el proyecto 🙄)

koajs/router: Router middleware for Koa:

Código y documentación de koa-router disponible en el repositorio oficial en GitHub

 [README](#)  [Code of conduct](#)  [MIT license](#)  [Security](#)  

@koa/router

Router middleware for [Koa](#). Maintained by [Forward Email](#) and [Lad](#).

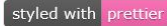
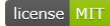
    

Table of Contents

- [Features](#)
- [Migrating to 7 / Koa 2](#)
- [Install](#)
- [Typescript Support](#)
- [API Reference](#)
- [Contributors](#)
- [License](#)

Features

- Express-style routing (`app.get` , `app.put` , `app.post` , etc.)
- Named URL parameters
- Named routes with URL generation
- Match routes with specific host

¿Cómo se ve Koa router?

```
const Koa = require('koa');
const Router = require('@koa/router');

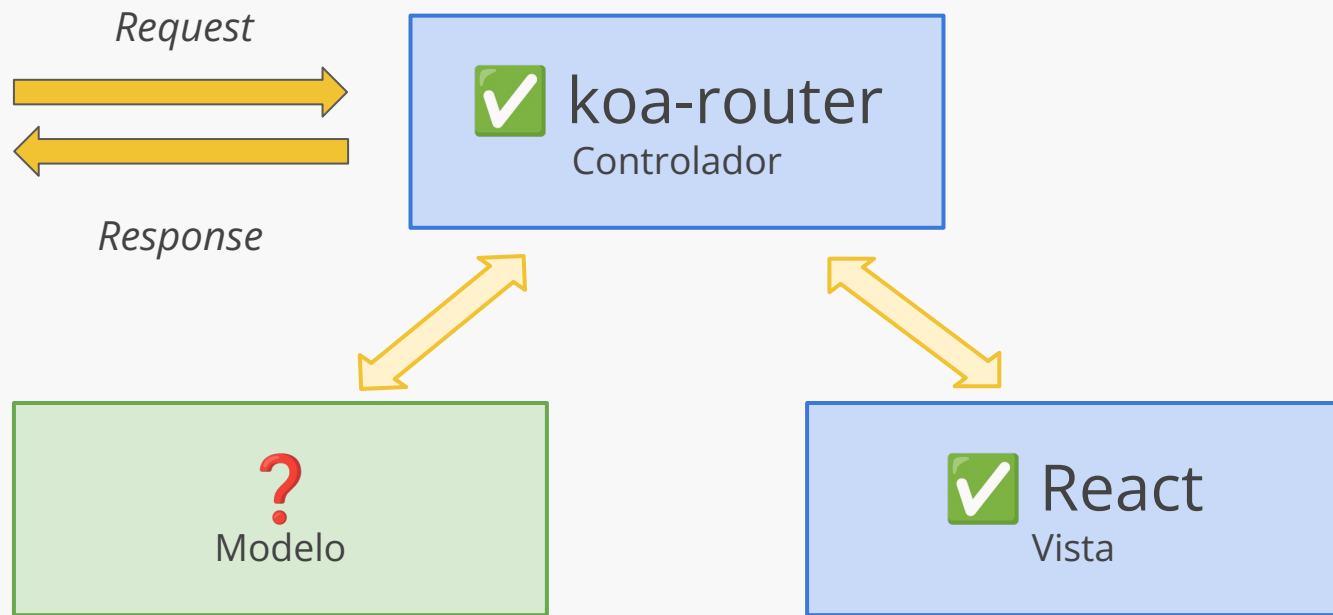
const app = new Koa();
const router = new Router();

router.get('/hello', (ctx, next) => {
  ctx.body = 'world!';
});

app
  .use(router.routes())
  .use(router.allowedMethods());

app.listen(3000);
```

¿Cómo se ve el proyecto del curso?



Agenda

Cliente y servidor

Frameworks web

Implementando MVC

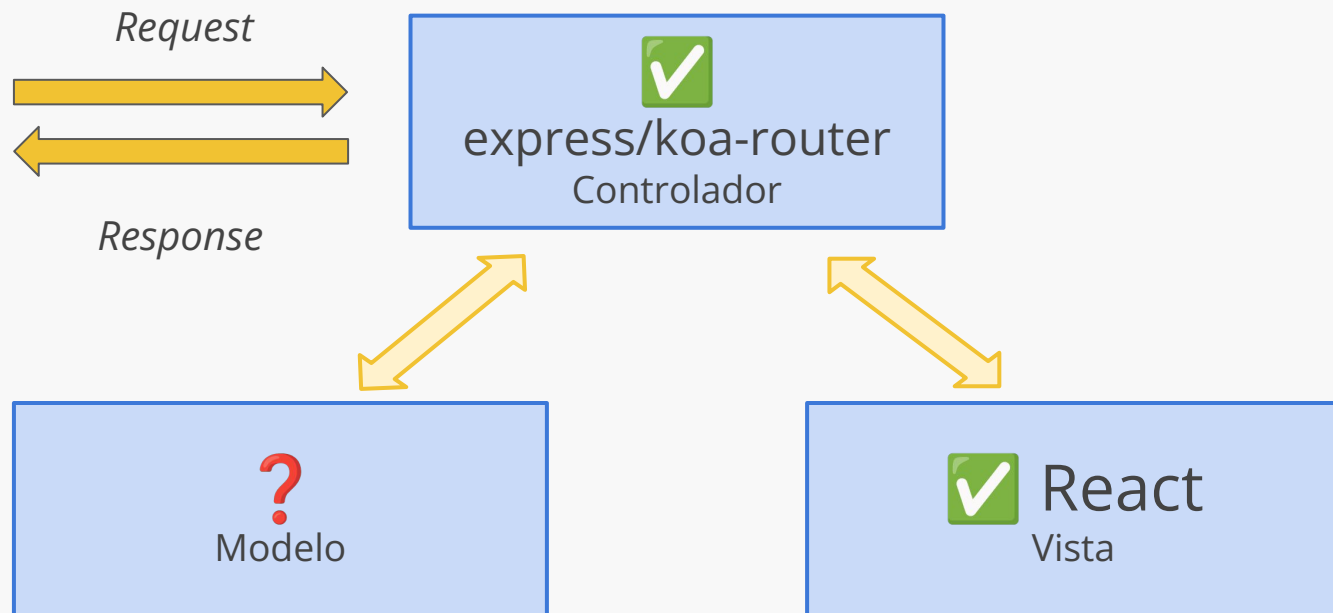
Vista

Controlador



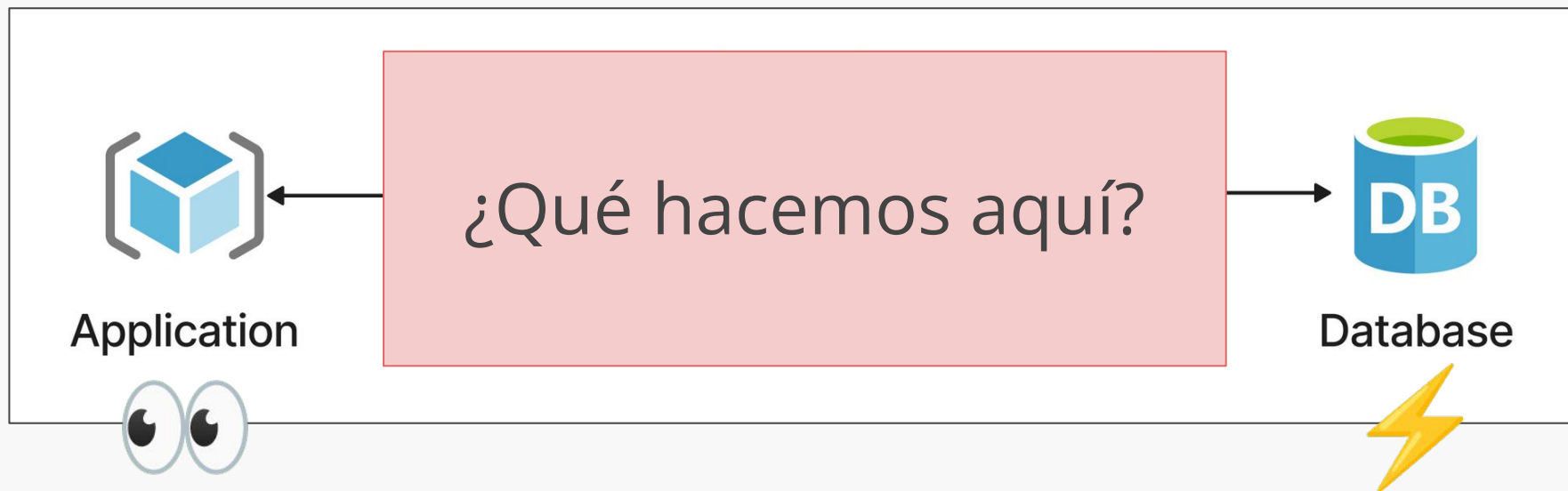
Modelo

¿Cómo se ve el proyecto del curso?



Objetivo: conectarnos a una base de datos

Lo que no queremos es tener que lidiar con SQL



Objetivo: conectarnos a una base de datos

1) Database Driver: Es una librería que nos permite conectarnos a una base de datos particular. En el caso de PostgreSQL la librería es pg

```
# Según la DB, se usa uno de los siguientes:  
$ npm install --save pg pg-hstore # Postgres  
$ npm install --save mysql2  
$ npm install --save mariadb  
$ npm install --save sqlite3  
$ npm install --save tedious # Microsoft SQL Server  
$ npm install --save oracledb # Oracle Database
```

Objetivo: conectarnos a una base de datos

3) ORM (Object-Relational Mapping): En este nivel de abstracción, buscamos separar el lenguaje de programación del trabajo directo con una base de datos. Estas librerías encapsulan el código requerido para trabajar con la DB , entregando una abstracción

```
1 // Consulta a base de datos SQL (a la C#)
2 let query = "SELECT * FROM users WHERE id = 10";
3 let result = context.Users.FromSqlRaw(sql).ToList();
4 let name = result[0]["first_name"];
5
6 // Consulta usando la API de un ORM
7 let user = repository.GetUser(10);
8 let firstName = user.GetFirstName();
9
10 // Otra alternativa, usando otra API
11 let user = User.Get(10)
```

Objetivo: conectarnos a una base de datos

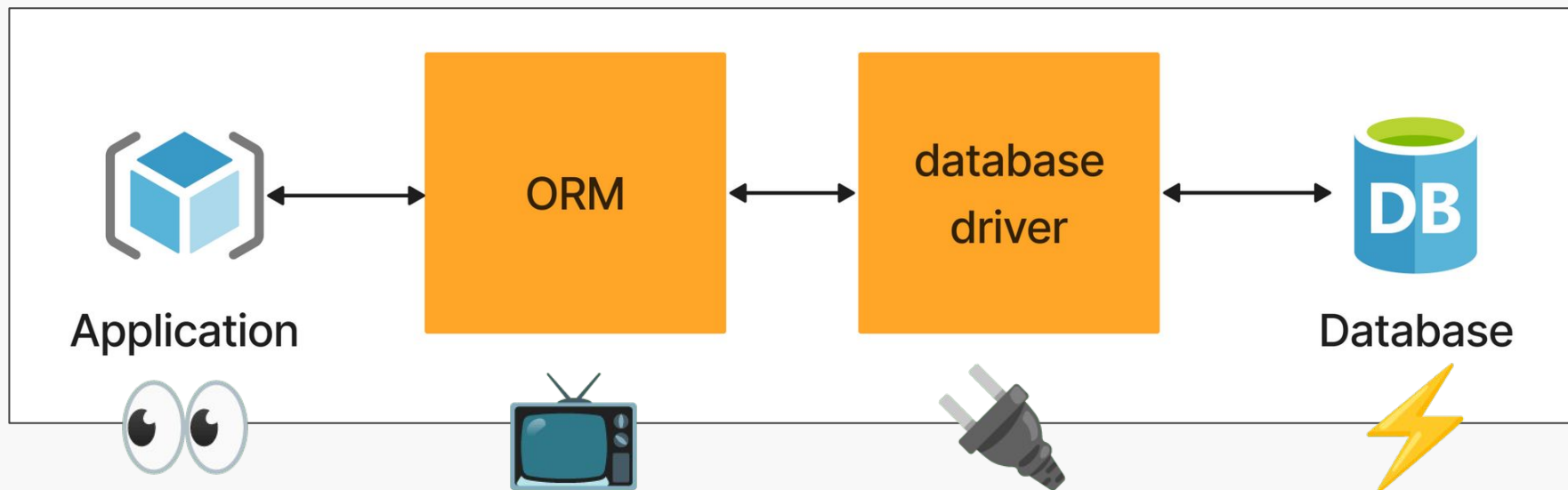
2) Query Builder: Capa intermedia entre el driver y el ORM que permite construir consultas con una sintaxis similar a la que usamos en SQL

PS: No es necesario usarlo en el proyecto

```
1 $orders = DB::table('orders')
2         ->select('department', DB::raw('SUM(price) as total_sales'))
3         ->groupBy('department')
4         ->havingRaw('SUM(price) > ?', [2500])
5         ->get();
6
```

Objetivo: conectarnos a una base de datos

Lo que no queremos es tener que lidiar con SQL



Objetivo: conectarnos a una base de datos

Lo que no queremos es tener que lidiar con SQL

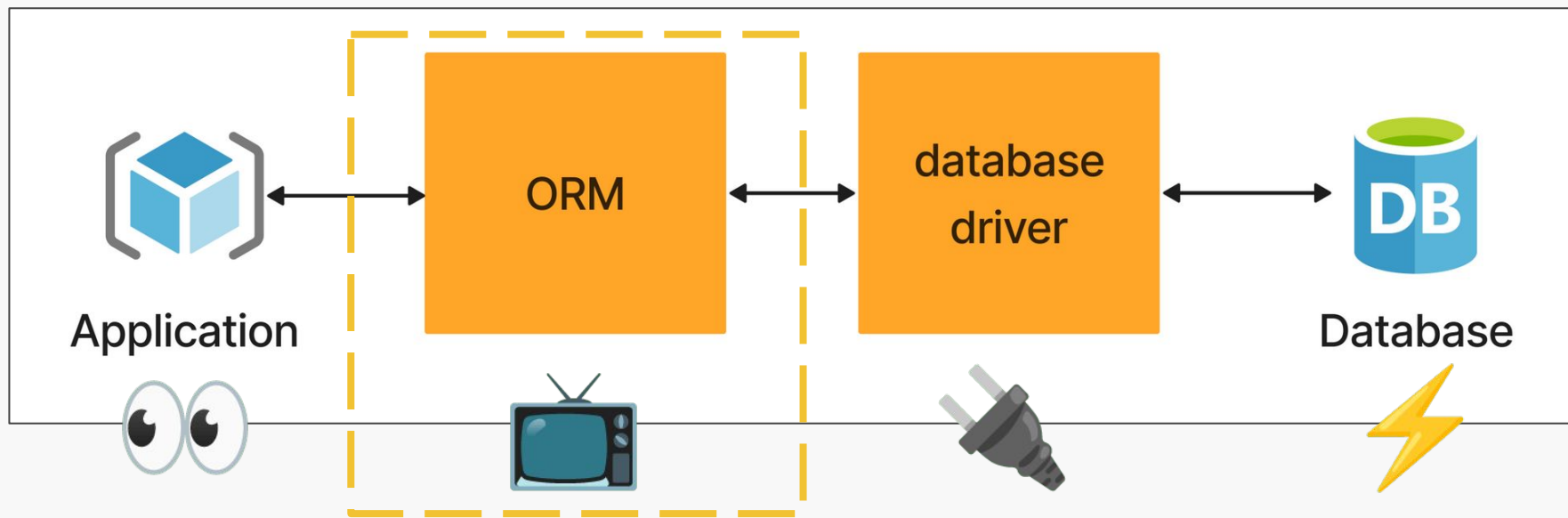


Figura: [Raw SQL vs. ORM: Who is the winner?](#)



Sequelize

Sequelize is a modern TypeScript and Node.js ORM for Oracle, Postgres, MySQL, MariaDB, SQLite and SQL Server, and more. Featuring solid transaction support, relations, eager and lazy loading, read replication and more.

[Getting Started](#)[API Reference](#)[Upgrade to v6](#)[Support us](#)

1. INSTALL DEPENDENCIES

```
npm install sequelize sqlite3  
# or  
yarn add sequelize sqlite3
```


Knex Query Builder:

Opcionalmente, en el proyecto pueden usar Knex como *query builder*

[Guide](#)[F.A.Q.](#)[Changelog](#)[GitHub](#)[PostgreSQL](#)[Search](#)[CTRL](#)[K](#)

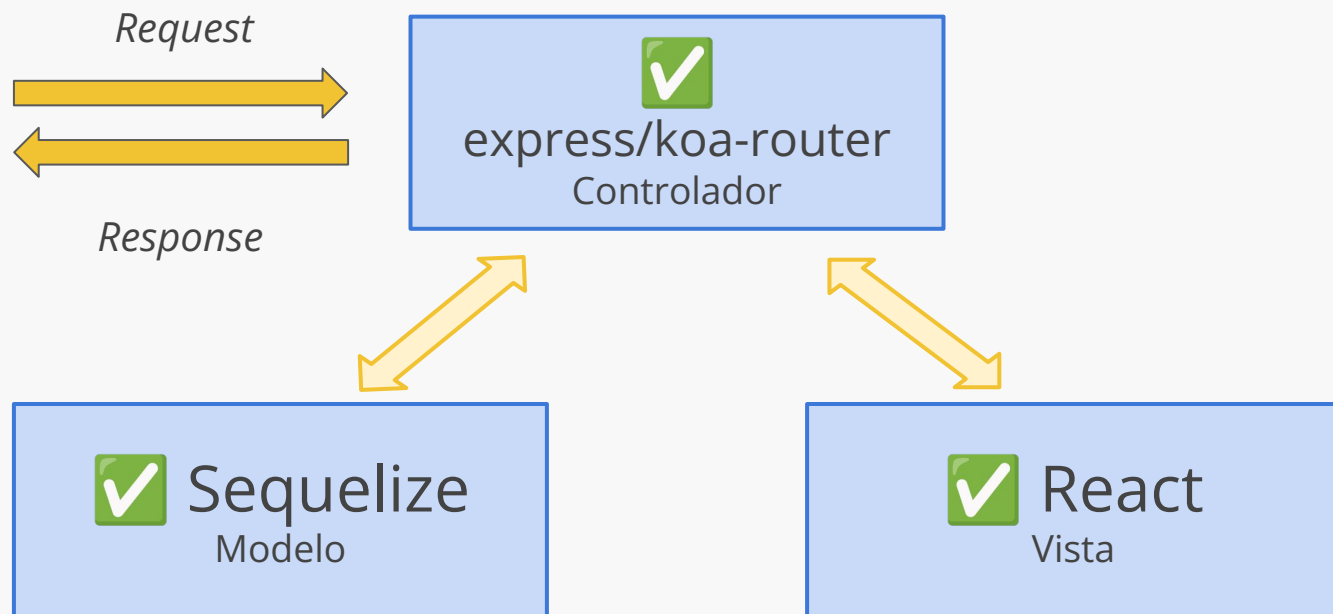
Knex.js

SQL query builder

[View guide](#)[Star on GitHub](#)

Knex.js (pronounced /kə'neks/) is a "batteries included" SQL query builder for PostgreSQL, CockroachDB, MSSQL, MySQL, MariaDB, SQLite3, Better-SQLite3, Oracle, and Amazon Redshift designed to be flexible, portable.

¿Cómo se ve el proyecto del curso?



Agenda

Cliente y servidor

Frameworks web

Implementando MVC



¿Qué se viene pronto?

Próximas fechas importantes: clases y ayudantías

	Clases y ayudantías						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 4 (24/3 - 30/3)		React		Exploración y consumo de APIs	Ayudantía: React		
Semana 5 (31/3 - 06/4)		Node.js		HOY	Ayudantía: Consumo de APIs		
Semana 6 (07/4 - 13/4)		Construcción de APIs		QA, CI & CD	Ayudantía: Construcción de APIs		
Semana 7 (14/4 - 20/4)		Fundamentos de front-end		Jueves Santo	Viernes Santo	Sábado Santo	Domingo de Resurrección
Semana 8 (21/4 - 27/4)		Charla Front-end (Florencia Valladares)		Diseño web en la práctica			

Próximas fechas importantes: evaluaciones

	Evaluaciones						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 4 (24/3 - 30/3)	Control 1				Inicio T1 React		
Semana 5 (31/3 - 06/4)				HOY	Fin T1 (React)		
Semana 6 (07/4 - 13/4)	Control 2				Inicio T2 API		
Semana 7 (14/4 - 20/4)				Jueves Santo	Viernes Santo	Sábado Santo	Domingo de Resurrección
Semana 8 (21/4 - 27/4)	UC: Sin eval.	UC: Sin eval.	Fin T2 (API)				

Clase 8

Desarrollando servidores

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra
aaossa@ing.puc.cl